

# US Patent & Trademark Office

## Patent Public Search | Text View

|                      |                      |
|----------------------|----------------------|
| United States Patent | 12399769             |
| Kind Code            | B2                   |
| Date of Patent       | August 26, 2025      |
| Inventor(s)          | Wang; Zhiying et al. |

### Systems and methods for distributed storage using storage codes with flexible number of nodes

#### Abstract

Processes and device configurations are provided for generating and using flexible storage codes. Methods and configurations described herein can reconstruct and recover data from a distributed network in the event of one or more failures or erasures. According to embodiments, a method includes determining the number of nodes and number of symbols of a flexible storage code, and reconstructing the information using the flexible storage code and determined numbers of nodes and symbols. The flexible storage code may be configured for a flexible number of nodes and symbols for information to use a subarray of symbols and nodes of a storage code array for information. Flexible storage codes may provide a framework for a reconstruction configuration using one or more of a flexible locally recoverable code (LRC), flexible partial maximum distance separable code (PMDSC), and flexible minimum storage regenerating code (MSRC).

|                   |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| <b>Inventors:</b> | <b>Wang; Zhiying (Irvine, CA), Li; Weiqi (Irvine, CA), Jafarkhani; Hamid (Irvine, CA)</b> |
| <b>Applicant:</b> | <b>The Regents of the University of California (Oakland, CA)</b>                          |
| <b>Family ID:</b> | <b>1000008780936</b>                                                                      |
| <b>Assignee:</b>  | <b>The Regents of the University of California (Oakland, CA)</b>                          |
| <b>Appl. No.:</b> | <b>17/865950</b>                                                                          |
| <b>Filed:</b>     | <b>July 15, 2022</b>                                                                      |

#### Prior Publication Data

|                            |                         |
|----------------------------|-------------------------|
| <b>Document Identifier</b> | <b>Publication Date</b> |
| US 20230037969 A1          | Feb. 09, 2023           |

#### Related U.S. Application Data

us-provisional-application US 63222218 20210715

#### Publication Classification

**Int. Cl.:** G06F11/10 (20060101); G06F11/07 (20060101)

**U.S. Cl.:**

**CPC** G06F11/0793 (20130101); G06F11/0709 (20130101); G06F11/10 (20130101);

#### Field of Classification Search

**CPC:** G06F (11/1088); G06F (11/1092); G06F (11/10); G06F (11/1076); G06F (11/1048)

#### References Cited

## U.S. PATENT DOCUMENTS

| Patent No.   | Issued Date | Patentee Name | U.S. Cl.    | CPC          |
|--------------|-------------|---------------|-------------|--------------|
| 9311194      | 12/2015     | Cypher        | N/A         | G06F 11/3433 |
| 10860424     | 12/2019     | Dhuse         | N/A         | G06F 11/1076 |
| 11474903     | 12/2021     | Cocagne       | N/A         | G06F 11/1076 |
| 2011/0208994 | 12/2010     | Chambliss     | 714/E11.088 | G06F 11/1092 |
| 2011/0289351 | 12/2010     | Rashmi        | 714/6.32    | H03M 13/616  |
| 2013/0205181 | 12/2012     | Blaum         | 714/763     | G06F 11/108  |
| 2015/0142863 | 12/2014     | Yuen          | 707/827     | H03M 13/13   |
| 2017/0179979 | 12/2016     | Hussain       | N/A         | G06F 11/1076 |
| 2018/0060169 | 12/2017     | Hussain       | N/A         | H03M 13/373  |
| 2019/0140667 | 12/2018     | Ren           | N/A         | H03M 13/3761 |
| 2020/0042390 | 12/2019     | Roberts       | N/A         | G06F 11/1092 |
| 2020/0244284 | 12/2019     | Blaum         | N/A         | H03M 13/2906 |
| 2021/0042036 | 12/2020     | Payton        | N/A         | G06F 3/067   |
| 2022/0091939 | 12/2021     | Dhuse         | N/A         | G06F 11/1088 |

## OTHER PUBLICATIONS

Li, Weiqi. Flexible Coding for Distributed Systems. University of California, Irvine, 2021. (Year: 2021). cited by examiner

---

*Primary Examiner:* Wilson; Yolanda L

*Attorney, Agent or Firm:* Stites & Harbison, PLLC

---

## Background/Summary

CROSS REFERENCE TO RELATED APPLICATION (1) This application claims priority to U.S. provisional application No. 63/222,218 titled SYSTEMS AND METHODS FOR DISTRIBUTED STORAGE USING STORAGE CODES WITH FLEXIBLE NUMBER OF NODES filed on Jul. 15, 2021, the content of which is expressly incorporated by reference in its entirety.

### FIELD

(1) The present disclosure generally relates to systems and networks for distributed storage of data over a plurality of nodes, and more particularly to processes and configurations for using storage codes with a flexible number of nodes.

### BACKGROUND

(2) In distributed systems, error-correcting codes are ubiquitous to achieve high efficiency and reliability. However, most codes have a fixed redundancy level. The number of failures varies over time, and as such, the fixed redundancy overcompensates for errors in practical systems and can result in latency. For example, when the number of failures is smaller than the designed redundancy level, the redundant storage nodes are not used efficiently. There is a desire for improvements in storage codes processes and configurations.

(3) Prior configurations have been proposed for error correction to minimize a cost function such as a linear combination of bandwidth, delay or the number of hops. Similarly, other processes have been proposed to reduce the bandwidth or to achieve the optimal field size. One drawback of existing processes may be the requirement of identifying the set of available nodes prior to computing. There is a desire for improvements which address one or more deficiencies of existing systems and improve reconstruction of data.

### BRIEF SUMMARY OF THE EMBODIMENTS

(4) Disclosed and described herein are systems, methods and configurations for using flexible storage codes to reconstruct data. According to embodiments, a method for reconstructing data using flexible storage codes includes determining, by a device, a node failure for received information using a storage code for the information, the received information received from at least one of a plurality of storage nodes, and determining, by the device, a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information. The method includes reconstructing, by the device, the received information using the determined number of nodes and the number of symbols the of flexible storage code.

(5) In one embodiment, determining a node failure includes identifying at least one node failure from the storage code for the information.

(6) In one embodiment, the plurality storage nodes are in a distributed network and information is received by the device with symbols encoded over a finite field into a number of nodes, the flexible storage code configured for a flexible number of nodes and symbols.

(7) In one embodiment, the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures.

(8) In one embodiment, the storage code is at least one of a maximum distance separable (MDS) storage code and a minimum storage regenerating (MSR) storage code.

- (9) In one embodiment, the flexible storage code is configured as a flexible locally recoverable code (LRC), wherein flexible LRC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , and code word symbol  $l$ , and wherein the code is parametrized by  $\{(R_{sub,j}, k_{sub,j}, \text{custom character}_{sub,j}): 1 \leq j \leq a\}$  that satisfies  $k_{sub,j} \cdot \text{custom character}_{sub,j} = k \cdot \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k_{sub,i} > k_{sub,2} \dots > k_{sub,a} = k$ ,  $\text{custom character}_{sub,a} = \text{custom character}$ , and
- (10)  $R_j = k_j + \frac{k_j}{r} - 1$ ,  $R_j = k_j + \frac{k_j}{r} - 1$ ,  
for single node failure recovery from a subset of nodes of the storage code.
- (11) In one embodiment, the flexible storage code is configured as a flexible partial maximum distance separable code (PMDSC) wherein, wherein flexible PMDSC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , code word symbol  $l$ , and extra symbol failures  $s$ , and wherein the code is parametrized by a set of  $\{(R_{sub,j}, k_{sub,j}, \text{custom character}_{sub,j}): 1 \leq j \leq a\}$  satisfying  $k_{sub,j} \cdot \text{custom character}_{sub,j} = k \cdot \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k_{sub,1} > k_{sub,2} \dots > k_{sub,a} = k$ ,  $\text{custom character}_{sub,a} = \text{custom character}$ , and  $R_{sub,j} = k_{sub,j}$  such that  $k_{sub,j}$  symbols are accessed in each node.
- (12) In one embodiment, the flexible storage code is configured as a flexible minimum storage regenerating code (MSRC), wherein repair bandwidth is a minimum amount of transmission required to repair a single node failure from all remaining nodes normalized by size of the node, wherein the repair bandwidth is bounded by the minimum storage regenerating (MSR) bound as
- (13)  $b \geq \frac{n-1}{n-k}$ .  
based on a code word length  $n$ , and dimension  $k$ .
- (14) In one embodiment, reconstructing the received information with the number of nodes and the number of symbols of a flexible storage code includes using a subarray of nodes and symbols of the storage code for the information.
- (15) In one embodiment, the method also includes updating the number of nodes and number of symbols of the flexible storage code for error correction of additional information.
- (16) Embodiments are also directed to a device configured to reconstruct data using flexible storage codes. The device includes a receiver configured to receive information from a distributed network and a controller. The controller configured to determine a node failure for received information using a storage code for the information, the received information received from at least one of a plurality of storage nodes, and determine a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information. The controller also configured to reconstruct the received information using the determined number of nodes and the number of symbols of flexible storage code.
- (17) Other aspects, features, and techniques will be apparent to one skilled in the relevant art in view of the following detailed description of the embodiments.

## Description

### BRIEF DESCRIPTION OF THE DRAWINGS

- (1) The features, objects, and advantages of the present disclosure will become more apparent from the detailed description set forth below when taken in conjunction with the drawings in which like reference characters identify correspondingly throughout and wherein:
- (2) FIG. 1 illustrates a system according to one or more embodiments;
- (3) FIG. 2 illustrates a process for using flexible storage codes to reconstruct data according to one or more embodiments;
- (4) FIG. 3 depicts a device configuration according to one or more embodiments;
- (5) FIG. 4 illustrates a graphical representation of a flexible storage code according to one or more embodiments;
- (6) FIG. 5 illustrates a graphical representation of a multiple-layer codes according to one or more embodiments;
- (7) FIG. 6 illustrates a graphical representation of a flexible LRC storage code according to one or more embodiments;
- (8) FIG. 7 illustrates a graphical representation of a flexible PMDS storage code according to one or more embodiments; and
- (9) FIG. 8 illustrates a graphical representation of latency according to one or more embodiments.

### DETAILED DESCRIPTION OF THE EXEMPLARY EMBODIMENTS

#### Overview and Terminology

- (10) According to embodiments, a system, methods and device configurations are provided for flexible storage codes that make it possible to recover the entire information through accessing a flexible number of nodes. According to embodiments, flexible storage codes are a class of error-correcting codes that can recover information from a flexible number of storage nodes. As a result, processes and devices make a better use of the available storage nodes in the presence of unpredictable node failures and reduce the data access latency. According to embodiments, use of flexible storage codes may include use of a framework for accessing and correction of data.
- (11) In distributed systems, error-correcting codes are ubiquitous to achieve high efficiency and reliability. Unlike codes that have a fixed redundancy level, embodiments provide solutions for systems and processes where the number of failures varies over time. When the number of failures is smaller than the designed redundancy level, the redundant storage nodes are not used efficiently. Embodiments provide flexible storage codes that make it possible to recover the entire information through accessing a flexible number of nodes.
- (12) Methods and systems are thus described in the various embodiments for using flexible storage codes. Methods and systems described herein may apply to distributed storage system in which storage infrastructure can split data across

multiple physical servers, and often across more than one data center. A cluster of storage units may provide information to one or more devices, with a mechanism for data synchronization and coordination between cluster nodes. In the event of a data error, such as one or more nodes of the system failing to provide data, flexible correction codes are provided that can account for the failure of nodes such that the number of symbols and/or parities may be selected for correction.

(13) According to embodiments, multiple constructions of Flexible MDS codes may be utilized for different application scenarios including error-correction codes, universally decodable matrices, secret sharing and private information retrieval. Embodiments provide a framework that can produce flexible storage codes for different code families including important types of storage codes, such as codes that efficiently recover from a single node failure, or codes that correct mixed types of node and symbol failures. Embodiments of the disclosure include a framework for flexible codes that can generate flexible storage codes given a construction of fixed (non-flexible) storage code.

(14) Flexible LRC (locally recoverable) codes allow information reconstruction from a variable number of available nodes while maintaining the locality property, providing efficient single node recovery. According to embodiments, for an  $(n, k, \text{custom character}, r)$  flexible LRC code parametrized by  $\{(R.\text{sub}.j, k.\text{sub}.j, \text{custom character.sub}.j): 1 \leq j \leq a\}$  that satisfies  $k.\text{sub}.j \text{ custom character.sub}.j = k \text{ custom character}, 1 \leq j \leq a, k.\text{sub}.1 > k.\text{sub}.2 > \dots > k.\text{sub}.a = k, \text{ custom character.sub}.a = \text{ custom character}$ , and

$$(15) R_j = k_j + \frac{k_j}{r} - 1;$$

each single node failure can be recovered from a subset of  $r$  nodes, while the total information is reconstructed by accessing  $\text{custom character.sub}.j$  symbols in  $R.\text{sub}.j$  nodes. Embodiments provide code constructions based on the optimal LRC code construction.

(16) Embodiments can be applied to using Flexible PMDS (partial MDS) codes are designed to tolerate a flexible number of node failures and a given number of extra symbol failures, desirable for solid-state drives due to the presence of mixed types of failures. We provide an  $(n, k, \text{custom character}, s)$  with a set of  $\{(R.\text{sub}.j, k.\text{sub}.j, \text{custom character.sub}.j): 1 \leq j \leq a\}$  satisfying  $k.\text{sub}.j \text{ custom character.sub}.j = k \text{ custom character}, 1 \leq j \leq a, k.\text{sub}.1 > k.\text{sub}.2 > \dots > k.\text{sub}.a = k, \text{ custom character.sub}.a = \text{ custom character}$ , and  $R.\text{sub}.j = k.\text{sub}.j$  such that when  $\text{custom character.sub}.j$  symbols are accessed in each node, we can tolerate  $n - R.\text{sub}.j$  failures and  $s$  extra symbol failures. Embodiments, construct flexible codes from the PMDS code.

(17) Embodiments can be applied to using Flexible MSR (minimum storage regenerating) codes as a type of flexible MDS codes such that a single node failure is recovered by downloading the minimum amount of information from the available nodes. Both vector and scalar codes are obtained by applying our flexible code framework to the MSR codes.

(18) The disclosure provides an analysis of benefits including latency analysis for flexible storage codes. It is demonstrated that flexible storage codes according to embodiments have a lower latency compared to the corresponding fixed codes.

(19) As used herein, the terms “a” or “an” shall mean one or more than one. The term “plurality” shall mean two or more than two. The term “another” is defined as a second or more. The terms “including” and/or “having” are open ended (e.g., comprising). The term “or” as used herein is to be interpreted as inclusive or meaning any one or any combination. Therefore, “A, B or C” means “any of the following: A; B; C; A and B; A and C; B and C; A, B and C”. An exception to this definition will occur only when a combination of elements, functions, steps or acts are in some way inherently mutually exclusive.

(20) Reference throughout this document to “one embodiment,” “certain embodiments,” “an embodiment,” or similar term means that a particular feature, structure, or characteristic described in connection with the embodiment is included in at least one embodiment. Thus, the appearances of such phrases in various places throughout this specification are not necessarily all referring to the same embodiment. Furthermore, the particular features, structures, or characteristics may be combined in any suitable manner on one or more embodiments without limitation.

#### EXEMPLARY EMBODIMENTS

(21) FIG. 1 illustrates a system according to one or more embodiments. System **100** is a distributed storage system including a plurality of storage devices, such as storage nodes **110.sub.1-n**, which may be configured to store and provide information to devices, such as device **105**. Storage nodes **110.sub.1-n**, may be configured to store information over a plurality of nodes, and provide information by way of one or more transmissions to a requesting device. According to embodiments, a framework for storage codes with a flexible number of nodes is provided. Information provided by the network may be transmitted with a storage code, the storage code including data to allow for error correction. The framework for flexible storage codes may be used by way of processes and device configurations. As described herein, device **105**, and/or one or more applications running on device **105**, may be configured to request data from distributed network **100**. Device **105** may be a computing device, mobile device or device in general including a processor. Storage nodes **110.sub.1-n** may be servers or other computing devices configured for network communication. According to embodiments, storage nodes may represent storage locations of a memory storage device. Storage nodes **110.sub.1-n** may transmit data by way of wired and/or wireless communication to device **105** and one or more other devices. In the event of a node failure or error in transmission of data, flexible storage nodes may be used for reconstruction. According to certain embodiments, processes and configurations discussed herein may be employed for storage of data by system **100**, such as data **120** provided by device **105** to storage nodes **110.sub.1-n**.

(22) As used herein, flexible storage codes may function to provide error correction. As further discussed herein the number of symbols and nodes used in a correction code may be based on the error. By way of example, two symbols may be accessed in three nodes, or three symbols may be access in two nodes depending on the error. Processes and device configurations described herein are configured to determine the number of symbols and nodes used in a correction.

(23) FIG. 2 illustrates a process for using flexible storage codes to reconstruct data according to one or more embodiments. According to embodiments, process **200** may be performed by a device (e.g., device **105**) requesting data and/or one or more storage devices of the distributed network (e.g., storage nodes **110.sub.1-n**) and one or more devices of a system (e.g., system

**100).** Process **200** may be performed by a device when information is received in a distributed network. Embodiments characterizing a distributed network may include information received from one or more sources or storage nodes. According to embodiments, operations may be performed for data received from one or more storage sources or devices. According to yet other embodiments, operations may be based on information stored with a storage code, such as least one of a maximum distance separable (MDS) storage code and a minimum storage regenerating (MSR) storage code.

(24) Process **200** may be initiated by a device determining that an error correction is required for information stored in a distributed network at block **205**. An error correction may be identified based on the failure or one or more nodes of the distributed network. Process **200** may be performed after information is received. Process **200** may optionally include receiving data, information, at block **220**. The device may identify that an error correction is required based on data received. Data received can include transmitted information and storage code information for the transmitted information. In a distributed network, data may be received from one or more storage nodes. Similarly, transmitted information may be transmitted in packets or groupings, such that an information stream may include one or more blocks or nodes of data. Determinations of node failure may be determined when data is not received from a storage device. Similarly, node failure may be based on a block of data that is part of a stream is not received from a storage device. Similarly, node failure may be determined when data is received with one or more errors. Embodiments can utilize storage code to recreate and/or regenerate damaged or missing data. Data with one or more errors, due to transmission errors, detection errors, etc., may be identified using one or more of symbols and parities in storage code associated with the data. With data transmission, speed and elimination of redundancy can improved device performance. Embodiments allow for operations to use flexible storage codes, such that the number of symbols and nodes of a storage code may be selected in order to reconstruct data. According to embodiments, process **200** can reconstruct data using one or more flexible storage codes described herein.

(25) At block **205**, process **200** may be performed by a device to determine a node failure for received information using a storage code for the information. The received information may be received from at least one of a plurality of storage nodes. The received information may be received in one or more blocks of code, the received blocks of code may be considered nodes for purposes of the disclosure. Node failure may be determined by identifying at least one node failure from the storage code using the storage code information. According to embodiments, one or more symbols encoded with the received information may be used to identify a node failure and/or node error. A plurality of storage nodes of a distributed network may provide information that is received by the device with symbols encoded over a finite field into a number of nodes. According to embodiments, information that can be broken up into blocks to include, each block being a node of information and where each block includes a storage code as symbols. Symbols of the blocks may also include parities to allow for error correction. According to embodiments and as described herein, a flexible storage code is used to identify and operate with a portion of transmitted symbols to provide a flexible number of nodes and symbols. Unlike error correction codes that use a fixed number of symbols, the fixed number of symbols being a subset of available symbols, processes described herein determine the number of symbols and nodes to use in received information for error correction. Determining the number and nodes of storage code may be based on the number of failures for received information.

(26) At block **210**, process **200** includes determining a number of nodes and number of symbols of a flexible storage code for error correction. According to embodiments, a system providing information may encode storage codes as information symbols over a finite field into a number of nodes. Nodes of the storage code relate to one or more nodes of data that are received of the information storage code. Storage code may be arranged as an array of codes including parameters for code word length, dimension, and size of each node. According to embodiments, the number of nodes and the number of symbols is determined based on number of failures and based on values of storage code information.

(27) Process **200** illustrates use of flexible storage codes, as a class of error-correcting codes that can recover information from a flexible number of storage nodes. According to embodiments, flexible storage codes relates to the use of elements within a storage code. By minimizing and/or selecting parameters of storage code, one or more of efficiency and speed of error correction may be improved. As a result, process **200** can make a better use of the available storage nodes in the presence of unpredictable node failures and reduce the data access latency. Process **200**, and embodiments herein, may characterize a storage system that encodes  $k$  custom character information symbols over a finite field  $\mathbb{F}$  custom character into  $n$  nodes, each of size  $\mathbb{F}$  custom character symbols. The code is parameterized by a set of tuples  $\{(R_{\text{sub}.j}, k_{\text{sub}.j}, \mathbb{F}_{\text{sub}.j}) : 1 \leq j \leq a\}$  satisfying  $k_{\text{sub}.1} \mathbb{F}_{\text{sub}.1} = k_{\text{sub}.2} \mathbb{F}_{\text{sub}.2} = \dots = k_{\text{sub}.a} \mathbb{F}_{\text{sub}.a}$  and  $k_{\text{sub}.1} > k_{\text{sub}.2} > \dots > k_{\text{sub}.a} = k$ ,  $\mathbb{F}_{\text{sub}.a} = \mathbb{F}$  custom character, such that the information symbols can be reconstructed from any  $R_{\text{sub}.j}$  nodes, each node accessing  $e_j$  symbols. In other words, the code allows a flexible number of nodes for decoding to accommodate the variance in the data access time of the nodes. Code constructions are presented for different storage scenarios, including LRC (locally recoverable) codes, PMDS (partial MDS) codes, and MSR (minimum storage regenerating) codes. As discussed herein analysis is provided for the latency of accessing information. According to other embodiments, the flexible storage code of process **200** may be configured as a code construction, such as one or more of a flexible locally recoverable code (LRC), flexible partial maximum distance separable code (PMDSC), and a flexible minimum storage regenerating code (MSRC).

(28) In one embodiment, the flexible storage code is configured as a flexible locally recoverable code (LRC), wherein flexible LRC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , and code word symbol  $l$ , and wherein the code is parametrized by  $\{(R_{\text{sub}.j}, k_{\text{sub}.j}, \mathbb{F}_{\text{sub}.j}) : 1 \leq j \leq a\}$  that satisfies  $k_{\text{sub}.j} \mathbb{F}_{\text{sub}.j} = k \mathbb{F}$  custom character,  $1 \leq j \leq a$ ,  $k_{\text{sub}.1} > k_{\text{sub}.2} > \dots > k_{\text{sub}.a} = k$ ,  $\mathbb{F}_{\text{sub}.a} = \mathbb{F}$  custom character, and

$$(29) R_j = k_j + \frac{k_j}{r} - 1,$$

for single node failure recovery from a subset of nodes of the storage code.

(30) In one embodiment, the flexible storage code is configured as a flexible partial maximum distance separable code (PMDSC) wherein, wherein flexible PMDSC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , code word symbol  $l$ , and extra symbol failures  $s$ , and wherein the code is parametrized by a set of  $\{(R.sub.j, k.sub.j, \text{custom character.sub.j}): 1 \leq j \leq a\}$  satisfying  $k.sub.j \text{custom character.sub.j} = k \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k.sub.1 > k.sub.2 \dots > k.sub.a = k$ ,  $\text{custom character.sub.a} = \text{custom character}$ , and  $R.sub.j = k.sub.j$  such that  $\text{custom character.sub.j}$  symbols are accessed in each node.

(31) In one embodiment, the flexible storage code is configured as a flexible minimum storage regenerating code (MSRC), wherein repair bandwidth is a minimum amount of transmission required to repair a single node failure from all remaining nodes normalized by size of the node, wherein the repair bandwidth is bounded by the minimum storage regenerating (MSR) bound as

$$(32) b \geq \frac{n-1}{n-k}.$$

based on a code word length  $n$ , and dimension  $k$ .

(33) At block **215**, process **200** includes reconstructing information using the flexible storage code and determined numbers of nodes and symbols of the flexible storage code using a subarray of nodes and symbols of the storage code for the information. Process **200** may include utilization of the number of symbols accessed and the number of nodes accessed. By way of example, two symbols may be accessed in three nodes or three symbols may be accessed in two nodes as is shown in FIG. **4**. It should be appreciated that other numerical combinations may be utilized based on the disclosure.

(34) Process **200** may include determining one or more additional nodes or symbols to use for decoding of other information. By way of example, after correction at block **215**, process **200** may include detection of a node failure at block **205**, such that particulars of the failure and number of failed nodes can be used for additional and/or subsequent error correction. As such, process **200** may include updating the number of nodes and number of symbols of the flexible storage code for error correction of additional information.

(35) At optional block **225**, process **200** may output of reconstructed information. Reconstructed information may provide the information requested from a distributed network, even in the event of a storage node failure or error in transmission.

(36) FIG. **3** depicts a device configuration according to one or more embodiments. Device **300** may relate to computing device (e.g., device **105**) configured to device configured for reconstructing data using flexible storage codes. According to one embodiment, receiver **300** includes controller **305**, memory **310** and communications module **315**. Controller **305** may relate to a processor or control device configured to execute one or more operations stored in memory **310**. Controller **305** may be coupled to memory **310** and communication module **315**. Communication module **315** may be configured to receive information from a distributed network. According to one embodiment, controller **305** is configured to determine that error correction is required for the information, determine a number of nodes and number of symbols of a flexible storage code, and reconstruct the information using the flexible storage code and determined numbers of nodes and symbols.

(37) FIG. **4** illustrates a graphical representation of a flexible storage code according to one or more embodiments. According to embodiments, flexible storage code **400** can include a plurality of information symbols **405.sub.1-n** and a plurality of parities **410.sub.1-n**. Flexible storage code **400** may be used to reconstruct information from a distributed network in the event of one more failures. Flexible storage code **400** may be an array code to describe information of a distributed network. According to embodiments, an  $(n, k, \text{custom character})$  field  $F$  is denoted by  $(C.sub.1, C.sub.2 \dots, C.sub.n)$ ,  $C.sub.i = (C.sub.1,i, C.sub.2,i, \dots, C.sub.\text{custom character},i).sup.T \in \text{custom character} \text{custom character}$ , where  $n$  is the code word length,  $k$  is the dimension, and  $\text{custom character}$  is the size of each node (or code word symbol) and is called the sub-packetization size. According to embodiments, processes may assume that for an  $(n, k, \text{custom character})$  code the entire information may be recovered by downloading all the symbols from any  $R$  nodes. According to embodiments, the download time of the slowest node among the  $R$  nodes is defined as the data access latency. In practical systems, the number of available nodes might be different over time and the latency of each node can be modelled as a random variable. Waiting for downloading all  $\text{custom character}$  symbols from exactly  $R$  nodes may result in a large delay. Hence, it is desirable to be able to adjust  $R$  and  $\text{custom character}$  according to the number of failures.

(38) Motivated by reducing the data access latency, embodiments provide flexible storage codes. According to embodiments, a flexible storage codes is an  $(n, k, \text{custom character})$  code that is parameterized by a given integer  $a$  and a set of tuples  $\{(R.sub.j, k.sub.j, \text{custom character.sub.j}): 1 \leq j \leq a\}$  that satisfies  $k.sub.j \text{custom character.sub.j} = k \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k.sub.1 > k.sub.2 \dots > k.sub.a = k$ ,  $\text{custom character.sub.a} = \text{custom character}$ , and if we take  $\text{custom character.sub.j}$  particular coordinates of each code word symbol, denoted by  $(C.sub.m.sub.1.sub.i, C.sub.m.sub.2.sub.i, \dots, C.sub.ml.sub.f.sub.i).sup.T \in \text{custom character} \text{custom character}.sup.j \in [n]$  where  $[n]$  is the set of integers smaller or equal to  $n$ , we can recover the entire information from any  $R.sub.j$  nodes.

(39) For example, flexible maximum distance separable (MDS) codes are codes satisfying the singleton bound for each  $k.sub.j$ , namely,  $\{(R.sub.j, k.sub.j, \text{custom character.sub.j}): 1 \leq j \leq a\}$ . FIG. **4** shows an example. It is easy to see that the flexible code in the example has a better expected latency than a fixed code with either  $k=2$  or  $3$ . In particular, each node can read and then send its three symbols one by one to the decoder (in practice, each symbol can be viewed as, for example, several Megabytes when multiple copies of the same code are applied). The flexible decoder can wait until 2 symbols from any 3 nodes, or 3 symbols from any 2 nodes are delivered, whose latency is the minimum of the two fixed codes.

(40) According to embodiments, flexible code **400** is an example of a  $(4; 2; 3)$  flexible MDS code. Flexible code **400** includes a plurality of information symbols **405.sub.1-n** (e.g.,  $C.sub.1,1; C.sub.1,2; C.sub.1,3; C.sub.2,1; C.sub.2,2; C.sub.2,3$ ) and in this example six (6) information symbols. Flexible code **400** includes a plurality of parities **410.sub.1-n** (e.g.,  $W.sub.1 = C.sub.1,1 + C.sub.1,2 + C.sub.1,3$ ,  $W'.sub.1 = C.sub.1,1 + 2 C.sub.1,2 + 3 C.sub.1,3$ , are the parities for  $C.sub.1,1, C.sub.1,2, C.sub.1,3$ , and  $W.sub.2 = C.sub.2,1 + C.sub.2,2 + C.sub.2,3$ ;  $W'.sub.2 = C.sub.2,1 + 2 C.sub.2,2 + 3 C.sub.2,3$  are the

parities for C.sub.2,1, C.sub.2,2, C.sub.2,3. FIG. 4 illustrates two scenarios, including a first scenario 415 (“Scenario 1”), and second scenario 425 (“Scenario 2”). The accessed symbols in each scenario are marked as 420 and 430.

$W'.sub.3=W'.sub.1+W'.sub.2$ ;  $W'.sub.4=W'.sub.1+2 W'.sub.2$  are the parities of  $W'.sub.1$  and  $W'.sub.2$ . In Scenario 1, shown as 415, all the information symbols are accessed, including information symbols 405.sub.1-n, the entire information can be obtained directly. In Scenario 2, shown as 425,  $W'.sub.1$  and  $W'.sub.2$ , shown as 410.sub.3 and 410.sub.4, are also the parities in Rows 1 and 2, respectively. Thus, according to embodiments, three symbols are obtained in the first two rows, and the entire information can be decoded.

(41) Framework for Flexible Storage Codes

(42) Embodiments are direct to providing flexible storage codes and to provide a framework for flexible codes to convert a fixed (e.g., non-flexible) code construction into a flexible one. For the purpose of illustration, a flexible MDS code example is used. Other types of code constructions are discussed herein.

(43) According to embodiments, a flexible storage code may be defined to incorporate a code word represented by an  $\text{custom character} \times n$  array over  $\text{custom character}$ , denoted as  $C \in (\text{custom character}^{\text{custom character}})$ , where  $n$  is called the code length, and  $\text{custom character}$  is called the sub-packetization. Each column in the code may correspond to a storage node. Embodiments can include selecting some fixed integers  $j \in [a]$ ,  $\text{custom character.sub.j} \in [\text{custom character}]$  ( $\text{custom character}^{\text{custom character}}$ ), and recovery thresholds  $R.sub.j \in [n]$ . Let the decoding columns  $R.sub.j.Math.[n]$  be a subset of  $R.sub.j$  columns, and the decoding rows  $I.sub.1, I.sub.2, \dots, I.sub.R.sub.j.Math.[\text{custom character}]$  be subsets of rows each with size  $\text{custom character.sub.j}$ . Denote by  $C[I.sub.1, I.sub.2, \dots, I.sub.R.sub.j]$  the  $\text{custom character}^{.sub.j \times R.sub.j}$  subarray of  $C$  that takes the rows  $I.sub.1$  in the first column of  $R.sub.j$ , the rows  $I.sub.2$  in the second column of  $R.sub.j$ ,  $\dots$ , and the rows  $I.sub.k$  in the last column of  $R.sub.j$ . The information will be reconstructed from this subarray. For flexible MDS codes, flexible MRS codes and flexible PMDS codes, the following is required:  $R.sub.j=k.sub.j$ . According to embodiments, for the above types of codes, the parameter  $R.sub.j$  may be omitted.

(44) For flexible LRC codes, the following is required:

$$(45) R_j = k_j + \frac{k_j}{n-k_j} + 1,$$

since the maximum distance is lower bounded by

$$(46) n - k_j - \frac{k_j}{n-k_j} + 2.$$

(47) The  $(n, k, \text{custom character})$  flexible storage code is parameterized by  $(R.sub.j, k.sub.j, \text{custom character.sub.j})_{j \in [a]}$ , for some positive integer  $a$ , such that  $k.sub.j \text{custom character.sub.j} = k \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k.sub.1 > k.sub.2 \dots > k.sub.a = k$ ,  $\text{custom character.sub.a} = \text{custom character}$ . The flexible storage code encodes  $k \text{custom character}$  information symbols over a finite field  $\text{custom character}$  into  $n$  nodes, each with  $\text{custom character}$  symbols. The code satisfies the following reconstruction condition for all  $j \in [a]$ : from any  $R.sub.j$  nodes, each node accesses a set of  $\text{custom character.sub.j}$  symbols, and we can reconstruct all the information symbols, for any  $j \in [a]$ . That is, the code is defined by

(48) an encoding function  $\epsilon: (\text{custom character}^{\text{custom character}})^{.sup.k} \rightarrow (\text{custom character}^{\text{custom character}})^{.sup.n}$ ,

(49) decoding functions  $D.sub.R.sub.j: (\text{custom character}^{\text{custom character}.sup.j})^{.sup.R.sub.j} \rightarrow (\text{custom character}^{\text{custom character}})^{.sup.k}$ , for all  $R.sub.j.Math.[n]$ ,  $|R.sub.j|=R.sub.j$ , decoding rows  $I.sub.1, I.sub.2, \dots, I.sub.R.sub.j.Math.[\text{custom character}]$ ,  $|I.sub.1|=|I.sub.2|= \dots =|I.sub.R.sub.j|= \text{custom character.sub.j}$ , which are dependent on the choice of decoding columns  $R.sub.j$ .

(50) The functions are chosen such that any information  $\text{custom character} \in (\text{custom character}^{\text{custom character}})^{.sup.k}$  can be reconstructed from the nodes in  $R.sub.j$ :

$$(51) D_{R_j}(\mathcal{E}(U)^{.Math.I_1, I_2, \dots, I_{R_j}}) = \text{custom character}.$$

(52) According to embodiments, a flexible MDS code is define as a flexible storage code, such that  $R.sub.j=k.sub.j$ .

(53) FIG. 4 illustrates an example of a  $(n, k, \text{custom character})=(4, 2, 3)$  flexible MDS code parameterized by  $(k.sub.j, \text{custom character.sub.j}) \in \{(3,2), (2,3)\}$ . In this example,  $k \text{custom character}=6$  information symbols are encoded over  $\text{custom character}$  to a code with  $n=4$ ,  $k=2$ ,  $\text{custom character}=3$ .

(54) Embodiments are also directed to decoding. From any  $k.sub.1=3$  nodes, each node accesses the first  $\text{custom character.sub.1}=2$  symbols: The first 2 rows form a single parity-check  $(4; 3; 2)$  MDS code, and thus a device can determine information symbols from any 3 out of 4 symbols in each row. From any  $k.sub.2=2$  nodes, each node accesses all the  $\text{custom character.sub.2}=3$  symbols. According to embodiments, operations can include first decoding  $W'.sub.1$  and  $W'.sub.2$  in the last row since the last row is a  $(4; 2; 1)$  MDS code. Then,  $(C.sub.1,1, C.sub.1,2, C.sub.1,3, W.sub.1, W'.sub.1)$  and  $(C.sub.2,1, C.sub.2,2, C.sub.2,3, W.sub.2, W'.sub.2)$  form two  $(5,3,1)$  MDS codes. According to embodiments, all the information symbols can be decoded from  $W'.sub.1 W'.sub.2$  and any 2 columns of the first 2 rows.

(55) According to embodiments, construction of a code may be based on a set of  $(n+k.sub.j-k.sub.a, k.sub.j, \text{custom character.sub.j}-\text{custom character.sub.j-1})$  codes, each code called a layer, such that  $k.sub.j \text{custom character.sub.j} = k \text{custom character}$ ,  $1 \leq j \leq a$ ,  $k.sub.1 > k.sub.2 \dots > k.sub.a = k$ ,  $\text{custom character.sub.a} = \text{custom character}$ ,  $\text{custom character.sub.0}=0$ . The first layer is encoded from the original information symbols and other layers are encoded from the “extra parities”. The intuition for the flexible reconstruction is that after accessing symbols from some layers, we can decode the corresponding information symbols, which is in turn extra parity symbols in an upper layer. Therefore, the decoder can afford accessing less code word symbols in the upper layer, resulting in a smaller recovery threshold.

(56) FIG. 5 illustrates a graphical representation of a multiple-layer codes according to one or more embodiments. FIG. 5

illustrates a graphical representation of storage codes **505**, extra parities **510**. Storage codes **500** include  $(n, k, \text{custom character})$  flexible storage codes parameterized by  $\{(k.\text{sub}.j, \text{custom character.sub}.j): 1 \leq j \leq a\}$ , such that  $k.\text{sub}.j \text{ custom character.sub}.j = k \text{ custom character}$ ,  $1 \leq j \leq a$ ,  $k.\text{sub}.1 > k.\text{sub}.2 \dots > k.\text{sub}.a = k$ ,  $\text{custom character.sub}.a = \text{custom character}$ .

(57) Each column, such as column **506**, of storage codes **500** is a node. Note that only the first  $n$  columns under storage nodes are stored, and the extra parities are auxiliary. Set  $\text{custom character.sub}.0 = 0$ . We have  $a$  layers, and Layer  $j$ ,  $j \in [a]$ , is an  $(n + k.\text{sub}.j - k.\text{sub}.a, k.\text{sub}.j, \text{custom character.sub}.j - \text{custom character.sub}.j-1)$  code

$[(C.\text{sub}.j,1, C.\text{sub}.j,2, \dots, C.\text{sub}.j,n), C'.\text{sub}.j,1, C'.\text{sub}.j,2, \dots, C'.\text{sub}.j, K.\text{sub}.j.\text{sub}.j - k.\text{sub}.a]$

where  $C.\text{sub}.j,i = [C.\text{sub}.j,1,i, C.\text{sub}.j,2,i, \dots, C.\text{sub}.j,k.\text{sub}.j.\text{sub}.j - k.\text{sub}.j.\text{sub}.i, \text{sup}.T \in \text{custom character custom character}.\text{sup}.j.\text{sup}. - \text{custom character.sub}.j-1, i \in [n]]$  are actually stored and  $C'.\text{sub}.j,i = [C'.\text{sub}.j,1,i, C'.\text{sub}.j,2,i, \dots, C'.\text{sub}.j, \text{custom character.sub}.j.\text{sub}.j - \text{custom character.sub}.j-1.\text{sub}.i, \text{sup}.T \in \text{custom character custom character.sub}.j.\text{sup}. - \text{custom character.sub}.j-1, i \in [k.\text{sub}.j - k.\text{sub}.a]]$ , such as **511**, are the auxiliary extra parities. The  $(n + k.\text{sub}.j - k.\text{sub}.a, k.\text{sub}.j, \text{custom character.sub}.j)$  code in the first layer is encoded from the  $k.\text{sub}.j$   $\text{custom character.sub}.j = k \text{ custom character}$  information symbols over  $\text{custom character custom character}$ , and the  $(n + k.\text{sub}.j - k.\text{sub}.a, k.\text{sub}.j, \text{custom character.sub}.j - \text{custom character.sub}.j-1)$  code in Layer  $j$ ,  $j \geq 2$ , is encoded from extra parities  $C'.\text{sub}.j,i$ , for  $j' \in [j-1]$ ,  $k.\text{sub}.j - k.\text{sub}.a + 1 \leq i \leq k.\text{sub}.j - k.\text{sub}.a$ . As a sanity check,  $\sum.\text{sub}.j = 1.\text{sup}.j-1 (k.\text{sub}.j-1 - k.\text{sub}.j) (\text{custom character.sub}.j' - \text{custom character.sub}.j'-1) = (k.\text{sub}.j-1 - k.\text{sub}.j) (\text{custom character.sub}.j-1 - \text{custom character.sub}.0) = k.\text{sub}.j (\text{custom character.sub}.j - \text{custom character.sub}.j-1)$  extra parities over IF are encoded into Layer  $j$ , which matches the code dimension in that layer. In this example,  $\text{custom character.sub}.0 = 0$  and  $k.\text{sub}.j-1 \text{ custom character.sub}.j-1 = k.\text{sub}.j \text{ custom character.sub}.j$ .

(58) According to embodiments, the construction discussed above can be applied to different kinds of codes. For an  $(n, k, \text{custom character})$  flexible MDS code, the entire information can be recovered from any  $k.\text{sub}.j$  nodes, each node accessing its first  $\text{custom character.sub}.j$  symbols.

(59) Embodiments also provide for the application of flexible codes to LRC (locally recoverable codes, PMDS (partial maximum distance separable) codes, and MSR (minimum storage regenerating) codes. These codes provide a flexible construction mechanism for the entire information, and either can reduce the single-failure repair cost (i.e., the number of helper nodes and the amount of transmitted information), or can tolerate a mixed types of failures. Application include failure protection in distributed storage systems and in solid-state drives.

(60) Flexible LRC

(61) An  $(n, k, \text{custom character}, r)$  LRC code is defined as a code with length  $n$ , dimension  $k$ , sub-packetization size  $\text{custom character}$  and locality  $r$ . Locality here means that for any single node failure or erasure, there exists a group of at most  $r$  available nodes (called helpers) such that the failure can be recovered from them. The minimum Hamming distance of an  $(n, k, \text{custom character}, r)$  LRC code is lower bounded in as

(62)  $d_{\min} \geq n - k - \frac{k}{r} + 2$ ,

and LRC codes achieving the bound are called optimal LRC codes. For simplicity, using  $(n, k, r)$  LRC codes to present  $(n, k, \text{custom character}, r)$  LRC codes with  $\text{custom character} = 1$ . According to embodiments, an LRC code may be optimal when  $(n, k, r)$  LRC codes encode  $k$  information symbols into

(63)  $0C = [C_{1,1}, C_{1,2}, \dots, C_{1,r+1}, \dots, C_{\frac{n}{r+1},1}, C_{\frac{n}{r+1},2}, \dots, C_{\frac{n}{r+1},r+1}]$

where each group

(64)  $\{C_{m,i} : i \in [r+1]\}, m \in [\frac{n}{r+1}]$

is an MDS code with dimension  $r$  and the whole code  $C$  has a minimum distance of

(65)  $n - k - \frac{k}{r} + 2$

such that all the information symbols can be decoded from any

(66)  $k + \frac{k}{r} - 1$

nodes.

(67) According to embodiments, the  $(n, k, \text{custom character}, r)$  flexible LRC code may be defined as parameterized by  $\{(R.\text{sub}.j, k.\text{sub}.j, \text{custom character.sub}.j): 1 \leq j \leq a\}$  as a flexible storage code, such that all the symbols of any node can be recovered by reading at most  $r$  other nodes, and

(68)  $R_j = k_j + \frac{k_j}{n-k_j} + 1$ .

(69) The above  $R.\text{sub}.j$  matches the minimum distance lower bound. As a result, our definition of flexible LRC code may imply optimal minimum Hamming distance when we consider all symbols at each node. In the flexible LRC code, first, extra groups are generated in each row. Then,  $r$  extra parities are chosen from each extra group and encoded into lower layers. During information reconstruction, extra parities and hence extra groups are recovered from lower layers, leading to a smaller number of required access.

(70) FIG. 6 illustrates a graphical representation of a flexible LRC storage code according to one or more embodiments. LRC code **600** is shown for an example construction of  $(n=12; k=4; \text{custom character}=3; r=2)$  flexible LRC code. In this code, Rows 1 and 2 are  $(n=12; k=6; r=2)$  LRC codes encoded from the information, and one extra group is generated in each row. Four extra parities may be taken from the extra groups, which are encoded into  $(n=12; k=4; r=2)$  LRC code in Row 3. In this example, there are 12 nodes and the nodes are evenly divided into four groups. Any single failed node can be recovered from the other two nodes in the same group. To recover the entire information, any eight nodes are required, each accessing the first two symbols, or any five nodes, each accessing all three symbols.

(71) Applying optimal LRC codes by groups to the construction determined herein for flexible codes, a  $(n, k,$

custom character, r) flexible LRC code may be parameterized by  $\{(R_{\text{sub},j}, k_{\text{sub},j}, \text{custom character.sub},j): 1 \leq j \leq a\}$ . In this example, n may be divisible by r+1 and all k.sub.j's are divisible by r here. The code is defined in custom character of size at least

$$(72) n + (k_1 - k_a) + \frac{r+1}{r}.$$

The resulting code turns out to be an  $(n, k_{\text{sub},j}, \text{custom character.sub},j, r)$  LRC code when custom character.sub,j symbols are accessed at each node. That is, for any single node failure, there exists a group of at most r helpers such that the failure can be recovered from them.

(73) According to embodiments, the construction of a flexible LRC code results in a flexible LRC code with locality r and  $\{(R_{\text{sub},j}, k_{\text{sub},j}, \text{custom character.sub},j): 1 \leq j \leq a\}$ .

(74) According to embodiments, encoding can be applied such that in a layer j, an optimal LRC code of

$$(75) (n + (k_j - k_a) \frac{r+1}{r}, k_j, r), j \in [a] \setminus \{0\}, = 0$$

may be applied to each row. The k custom character information symbols in the custom character.sub,j rows of Layer 1, and the remaining rows may be encoded from extra parities.

(76) Embodiments include choosing the n stored symbols and the k.sub.j-k.sub.a extra parities in each row. In a

$$(77) (n + (k_j - k_a) \frac{r+1}{r}, k_j, r)$$

LRC code, there may be

$$(78) \frac{n}{r+1} + \frac{k_j - k_a}{r}$$

groups. The

$$(79) \frac{n}{r+1} + \frac{k_j - k_a}{r}$$

groups are selected containing n symbols. The, n stored symbols in each row form an  $(n, k_{\text{sub},j}, r), j \in [a]$  optimal LRC code. In the remaining

$$(80) 0 \frac{k_j - k_a}{r}$$

groups, r nodes may be selected in each group which contains k.sub.j-k.sub.a nodes, as extra parities. Since all the information symbols are encoded in Layer 1, information symbols can be decoded if enough dimensions to decode Layer 1 are obtained.

(81) Embodiments allow for decoding all information symbols from any

$$(82) R_j = k_j + \frac{k_j}{r} - 1, j \in [a]$$

nodes, each nodes accesses the first custom character.sub,j symbols.

(83) From layer j, since each row of it is part of the

$$(84) (n + (k_j - k_a) \frac{r+1}{r}, k_j, r)$$

optimal LRC code, the layer can be decoded from R.sub.j nodes by a property of the optimal LRC codes.

(85) By way of example, given that  $1 \leq j_{\text{sup},|} \leq j$  and that Layers  $j_{\text{sup},|}, j_{\text{sup},|}+1, \dots, j$  are decoded, the construction can provide that all the extra parities in Layer  $j_{\text{sup},|} - 1$  are included as the information symbols in Layers  $j_{\text{sup},|}, j_{\text{sup},|}+1, \dots, j$  and are decoded. Also, we know from the encoding part that the extra parities in Layer  $j_{\text{sup},|} - 1$  include of the r parity symbols in each group of the

$$(86) (n + (k_{j-1} - k_a) \frac{r+1}{r}, k_{j-1}, r)$$

optimal LRC codes. Thus, according to the locality, the remaining symbol in all

$$(87) (k_{j-1} - k_j) \frac{r+1}{r}$$

groups in each row can be reconstructed. Therefore, we get additional

$$(88) (k_{j-1} - k_j) \frac{r+1}{r}$$

symbols in each row of Layer  $j-1$  from the extra parities. Together with the R.sub.j nodes we accessed in each row of Layer  $j-1$ , we get R.sub.j'-1 symbols and, we are able to decode Layer  $j'-1$ .

(89) Locality: Since each row is encoded as a LRC code with locality r, every layer and the entire code also have locality r.

(90) By way of example, the following parameters may be set as  $(n, k, l, r) = (12, 4, 3, 2)$ ,  $(R_{\text{sub},1}, k_{\text{sub},1}, \text{custom character.sub},1) = (8, 6, 2)$ ,  $(R_{\text{sub},2}, k_{\text{sub},2}, \text{custom character.sub},1) = (5, 4, 2)$ . The code is defined over custom character  $= \text{GF}(2^{\text{sup},4}) = \{0, 1, \alpha, \dots, \alpha^{\text{sup},14}\}$ , where  $\alpha$  is a primitive element of the field. In total there may be k custom character information symbols, these may be assumed as u.sub.1,0, u.sub.1,1,  $\dots$ , u.sub.1,5, u.sub.2,0, u.sub.2,1,  $\dots$ , u.sub.2,5. The example is based on the optimal LRC code constructions. The construction is shown below, each column is a node with 3 symbols:

$$(91) \begin{bmatrix} C_{1,1,1} & C_{1,1,2} & C_{1,1,3} & C_{1,1,4} \\ C_{1,2,1} & C_{1,2,2} & C_{1,2,3} & C_{1,2,4} \\ C_{2,1,1} & C_{2,1,2} & C_{2,1,3} & C_{2,1,4} \end{bmatrix},$$

where every entry in Row m will be constructed as f.sub.m(x) for some polynomial f.sub.m( $\cdot$ ) and some field element x as below, m=1, 2, 3.

(92) According to embodiments, evaluation points are divided into 4 groups as  $x \in A = \{A_{\text{sub},1} = \{1, \alpha^{\text{sup},5}, \alpha^{\text{sup},10}\}, A_{\text{sub},2} = \{\alpha, \alpha^{\text{sup},6}, \alpha^{\text{sup},11}\}, A_{\text{sub},3} = \{\alpha^{\text{sup},2}, \alpha^{\text{sup},7}, \alpha^{\text{sup},12}\}, A_{\text{sub},4} = \{\alpha^{\text{sup},3}, \alpha^{\text{sup},8}, \alpha^{\text{sup},13}\}\}$ . In addition,  $A_{\text{sub},5} = \{\alpha^{\text{sup},4}, \alpha^{\text{sup},9}, \alpha^{\text{sup},15}\}$  may be set as the evaluation points group for the extra parities. By defining  $g(x) = x^{\text{sup},3}$ , g(x) is a constant for each group  $A_{\text{sub},i}$ ,  $i \in [5]$ . Then the first two rows are encoded with  $f_{\text{sub},m}(x) = (u_{\text{sub},m,0} + u_{\text{sub},m,1}g(x) + u_{\text{sub},m,2}g^{\text{sup},2}(x)) + x(u_{\text{sub},m,3} + u_{\text{sub},m,4}g(x) + u_{\text{sub},m,5}g^{\text{sup},2}(x))$ , m=1,2.

(93) The last row is encoded with  $f_{\text{sub},3}(x) = (f_{\text{sub},1}(\alpha^{\text{sup},4}) + f_{\text{sub},1}(\alpha^{\text{sup},9})g(x)) + x(f_{\text{sub},2}(\alpha^{\text{sup},4}) + f_{\text{sub},2}(\alpha^{\text{sup},9})g(x))$ .

(94) For each group, since  $g(x)$  is a constant,  $f_{\text{sub},m}(x)$ ,  $m \in [3]$  can be viewed as a polynomial of degree 2. Any single failure can be recovered from the other 2 available nodes evaluated by the points in the same group. The locality  $r=2$  is achieved.

(95) Noticing that  $f_{\text{sub},1}(x)$  and  $f_{\text{sub},2}(x)$  are polynomials of degree 7, all information symbols can be reconstructed from the first  $\text{custom character.sub.1}=2$  rows of any  $R_{\text{sub},1}=8$  available nodes.

(96) Moreover,  $f_{\text{sub},3}(x)$  has degree 4. With  $R_{\text{sub},2}=5$  available nodes, we can first decode  $f_{\text{sub},1}(\alpha_{\text{sup},4})$ ,  $f_{\text{sub},1}(\alpha_{\text{sup},9})$ ,  $f_{\text{sub},2}(\alpha_{\text{sup},4})$ ,  $f_{\text{sub},12}(\alpha_{\text{sup},9})$  in row 3. Then,  $f_{\text{sub},1}(\alpha_{\text{sup},14})$ ,  $f_{\text{sub},2}(\alpha_{\text{sup},14})$  can be decoded due to the locality  $r=2$ . At last, together with the 5 other evaluations of  $f_{\text{sub},1}(x)$  and  $f_{\text{sub},2}(x)$  obtained in Rows 1 and 2, we are able to decode all information symbols

(97) Flexible PMDS codes

(98) According to embodiments, flexible storage codes may be configured for PMDS codes. PMDS codes may be used to overcome mixed types of failures in Redundant Arrays of Independent Disks (RAID) systems using Solid-State Drives (SSDs). A code consisting of an  $\text{custom character } x$  array is an  $(n, k, \text{custom character}, s)$  PMDS code if it can tolerate  $n-k$  node or column failures and  $s$  additional arbitrary symbol failures in the code.

(99) By way of example, letting  $\text{custom character.sub.0}=0$  and  $\{(k_{\text{sub},j}, \text{custom character.sub.}j): 1 \leq j \leq a\}$  satisfying requirements of a flexible code. We define an  $(n, k, \text{custom character}, s)$  flexible PMDS code parameterized by  $\{(k_{\text{sub},j}, \text{custom character.sub.}j): 1 \leq j \leq a\}$  such that any row in  $[\text{custom character.sub.}j-1+1, \text{custom character.sub.}j]$  is an  $(n, k_{\text{sub},j})$  MDS code, and from the first  $\text{custom character.sub.}j$  rows, we can reconstruct the entire information if there are up to  $n-k_j$  node failures and up to  $s$  additional arbitrary symbol failures,  $1 \leq j \leq a$ . As mentioned, for PMDS codes,  $R_{\text{sub},j}=k_{\text{sub},j}$ .

(100) Note that different from above, the number of information symbols for a flexible PMDS code is at most  $\text{custom character}-s \cong K$ .

(101) FIG. 7 illustrates a graphical representation of a flexible PMDS storage code according to one or more embodiments. Flexible PMDS code **700** may be a  $(5; 3; 4; 2)$  flexible PMDS code with  $\{(k_{\text{sub},1}, \text{custom character.sub.1}), (k_{\text{sub},2}, \text{custom character.sub.2})\} = \{(4, 3), (3, 4)\}$ . Assuming there is only a value of failures, such as “\*” as failures, the first four nodes can be used to decode, each node accessing the first three symbols. If both “\*” and “4” are failures, we can decode from Nodes 1; 3; 4, each node accessing 4 symbols. In both cases, the remaining  $K=k_{\text{sub},j}-s=10$  symbols are independent and sufficient to reconstruct the entire information. The details of the encoding and decoding for this construction are presented herein.

(102) According to embodiments, a flexible PMDS code can encode the information using Gabidulin code into auxiliary symbols, which are allocated to each layer according to  $k_{\text{sub},j}$ ,  $j \in [a]$ . MDS codes with different dimensions are then applied to each row, thus ensuring flexible information reconstruction. Configurations may utilize a general construction of PMDS codes for any  $k$  and  $s$  using Gabidulin codes. In addition to the construction, embodiments include applying the construction to flexible PMDS codes.

(103) An  $(N, K)$  Gabidulin code over the finite field  $\text{custom character} = \text{GF}(q_{\text{sup},L})$ ,  $L \geq N$  is defined by the polynomial  $f(x) = \sum_{i=0}^{K-1} u_{\text{sub},i} x^{\alpha_{\text{sub},i}}$ , where  $u_{\text{sub},i} \in \text{custom character}$ ,  $i=0, 1, \dots, K-1$  is the information symbol. The  $N$  code word symbols are  $f(\alpha_{\text{sub},1}), f(\alpha_{\text{sub},2}), \dots, f(\alpha_{\text{sub},N})$  where the  $N$  evaluation points  $\alpha_{\text{sub},1}, \dots, \alpha_{\text{sub},N}$  are linearly independent over  $\text{GF}(q)$ . From any  $K$  independent evaluation points over  $\text{GF}(q)$ , the information can be recovered.

(104) By way of example, the  $(n, k, \text{custom character}, s)$  code word may be an  $\text{custom character} \times n$  matrix over  $\text{custom character} = \text{GF}(q_{\text{sup},k_{\text{sub},j}})$  such as:

$$(105) \begin{bmatrix} C_{1,1} & C_{1,2} & \dots & C_{1,n} \\ C_{2,1} & C_{2,2} & \dots & C_{2,n} \\ \dots & \dots & \ddots & \dots \\ C_{\ell,1} & C_{\ell,2} & \dots & C_{\ell,n} \end{bmatrix}$$

where each column is a node. By setting  $K=k_{\text{sub},j}-s$ ,  $C_{\text{sub},m,i} \in \text{custom character}$ ,  $m \in [\text{custom character}]$ ,  $i \in [k]$  are the  $K+s$  code word symbols from a  $(K+s, K)$  Gabidulin code, and for each row  $\text{sup.}m$ ,  $m \in [\text{custom character} \text{sup.}]$ .

$[C_{\text{sub},m,k+1}, \dots, C_{\text{sub},m,n}] = [C_{\text{sub},m,1}, \dots, C_{\text{sub},m,k}] G_{\text{sub},\text{MDS}}$

where  $G_{\text{sub},\text{MDS}}$  is  $k \times (n-k)$  encoding matrix of an  $(n, k)$  systematic MDS code over  $\text{GF}(q)$  that generates the parity.

Considering that  $t_{\text{sub},m}$  symbols in row  $m$ ,  $m \in [\text{custom character}]$  are equivalent to evaluations of  $f(x)$  with

$$(106) \text{Math.}_{m=1}^{\ell} \min(t_m, k)$$

evaluation points that are linearly independent over  $\text{GF}(q)$ . Thus, with any  $n-k$  node failures and  $s$  symbol failures, we have  $t_{\text{sub},m} \leq k$  and

$$(107) \text{Math.}_{m=1}^{\ell} \min(t_m, k) = \text{Math.}_{m=1}^{\ell} t_m = \ell k - s = K.$$

(108) Then, with the  $K$  linearly independent evaluations of  $f(x)$ , we can decode all information symbols. Next, we show how to construct flexible PMDS codes. Rather than generating extra parities as constructions above, the main idea here is that we divide our code into multiple layers, and each layer applies a construction with a different dimension.

(109) Embodiments may construct a  $(n, k, \text{custom character}, s)$  flexible PMDS code over  $\text{GF}(q_{\text{sup},N})$  parameterized by  $\{(k_{\text{sub},j}, \text{custom character.sub.}j): 1 \leq j \leq a\}$  satisfying requirements of a flexible code and with an  $(N, K)$  Gabidulin code over,

$$(110) \text{GF}(q^N), N = \text{Math.}_{j=1}^a k_j \text{Math.}_{\ell_j - \ell_{j-1}}, K = \ell k - s$$

and a set of  $(n, k_{\text{sub},j})$  systematic MDS codes over  $\text{GF}(q)$ .

(111) With respect to encoding:  $C_{\text{sub},j,m,i}$  is the symbol in the  $m$ -th row of Layer  $\text{sub},j$ , and in the  $i$ -th node,  $j \in [a]$ ,  $m, \text{sub},j \in \text{custom character.sub},j$ ,  $i \in [n]$ . We first encode the  $K$  information symbols using the  $(N,K)$  Gabidulin code. Then, we set the first  $k_{\text{sub},j}$  code word symbols in each row:  $C_{\text{sub},j,m,i}$ ,  $j \in [a]$ ,  $m, \text{sub},j \in \text{custom character.sub},j$ ,  $i \in [k_{\text{sub},j}]$ , as the code word symbols in the  $(N,K)$  Gabidulin code. The remaining  $n - k_{\text{sub},j}$  code word symbols in each row are  $[C_{\text{sub},j,m,\text{sub},j,\text{sub},j+1}, \dots, C_{\text{sub},j,m,\text{sub},j,\text{sub},n}] = [C_{\text{sub},j,m,\text{sub},j,\text{sub},1}, \dots, C_{\text{sub},j,m,\text{sub},j,\text{sub},k_{\text{sub},j}}] G_{\text{sub},n,k}$ , where  $G_{\text{sub},n,k}$  is the encoding matrix (to generate the parity check symbols) of the  $(n, k_{\text{sub},j})$  systematic MDS code over  $GF(q)$ .

(112) With respect to decoding: for  $n - k_{\text{sub},j}$  failures, we access the first  $\text{custom character.sub},J$  rows (the first  $J$  layers) from each node. The code structure in each layer is similar to the general PMDS code, and a union of  $t_{\text{sub},mj}$  symbols in Row  $m, \text{sub},j$  of Layer  $j$ ,  $j \leq J$ , they are equivalent to evaluations of  $f(x)$  with

$$(113) \text{Math}_{j=1}^J \text{Math}_{m_j=1}^{\ell_j - \ell_{j-1}} \min(t_{m_j}, k)$$

linearly independent points over  $GF(q)$  in  $GF(q.\text{sup},N)$ . Thus, with  $n - k_{\text{sub},J}$  node failures and  $s$  symbol failures, we have  $t_{\text{sub},m,\text{sub},j} \leq K_{\text{sub},J} \leq K_{\text{sub},j}$  for  $j \in [J]$ , and

$$(114) \text{Math}_{j=1}^J \text{Math}_{m_j=1}^{\ell_j - \ell_{j-1}} \min(t_{m_j}, k_j) = \text{Math}_{j=1}^J \text{Math}_{m_j=1}^{\ell_j - \ell_{j-1}} t_{m_j} = \ell_J k_J - s = K$$

(115) Then, the information symbols can be decoded from  $K$  linearly independent evaluations of  $f(x)$ .

(116) Flexible MSR Codes

(117) Regarding flexible MSR codes, embodiments can include a number of parity nodes is denoted by  $r = n - k$ . The repair bandwidth is defined as the amount of transmission required to repair a single node erasure, or failure, from all remaining nodes (called helper nodes), normalized by the size of the node. For an  $(n, k)$  MDS code, the repair bandwidth is bounded by the minimum storage regenerating (MSR) bound as

$$(118) b \geq \frac{n-1}{n-k}.$$

(119) An MDS code achieving the MSR bound is called an MSR code. For MSR vector codes, each symbol is a vector. As one of the most popular codes in practical systems, Reed-Solomon (RS) code, where each symbol is a scalar.

(120) As discussed herein, a set of MDS codes can recover the information symbols by any pair  $(k_{\text{sub},j}, \text{custom character.sub},j)$ , which means that for the first  $\text{custom character.sub},j$  symbols in each node, code according to embodiments is an  $(n, k_{\text{sub},j}, \text{custom character.sub},j)$  MDS code. In addition, we require the optimal repair bandwidth property for flexible MSR codes. A flexible MSR code is defined to be a flexible storage code as such that  $R_{\text{sub},j} = k_{\text{sub},j}$ , and a single node failure is recovered using a repair bandwidth satisfying the MSR bound.

(121) According to embodiments, codes in this section may be similar to constructions according to other embodiments above, with additional restrictions on the parity check matrices and the extra parities. A key point here may be that extra parities and the information symbols in lower layers are exactly the same and they also share the same parity check sub-matrix. To repair the failed symbol with smallest bandwidth, the extra parities are viewed as additional helpers and the required information can be obtained for free from the repair of the lower layers.

(122) An example is provided to illustrate 2 layers and then present our constructions based on vector and scalar MSR codes, respectively.

(123) Constructing an  $(n, k, \text{custom character}) = (4, 2, 3)$  flexible MSR code parameterized by  $(k_{\text{sub},1}, \text{custom character.sub},1) = (3, 2)$  and  $(k_{\text{sub},2}, \text{custom character.sub},2) = (2; 3)$ . The reconstruction of the entire information and the repair bandwidth are described above. Initially, the following is set  $\text{custom character} = GF(2.\text{sup},2) = \{0, 1, \beta, \beta.\text{sup},2 = 1 + \beta\}$ , where  $\beta$  is a primitive element of  $GF(2.\text{sup},2)$ . Our construction is based on the following  $(4; 2; 2)$  MSR vector code over  $\text{custom character.sup},2$  with parity check matrix

$$(124) H = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & h_{1,4} \\ h_{2,1} & h_{2,2} & h_{2,3} & h_{2,4} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

where each  $h_{\text{sub},i,j}$  is a  $2 \times 2$  matrix over  $\text{custom character}$ . A code word symbol  $c_{\text{sub},i}$  is  $\text{custom character.sup},2$ ,  $i = 1, 2, 3, 4$ , and the code word  $[c_{\text{sub},1.\text{sup},T}, c_{\text{sub},2.\text{sup},T}, c_{\text{sub},3.\text{sup},T}, c_{\text{sub},4.\text{sup},T}].\text{sup},T \in \text{custom character.sup},2$  in the null space of  $H$ . A  $(4; 2)$  MDS code may be checked using any two code word symbols suffice to reconstruct the entire information. The repair matrix is defined as

$$(125) S_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, S_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, S_3 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}, S_4 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

(126) When node  $* \in \{1, 2, 3, 4\}$  fails, the node  $c^*$  can be repaired by equations  $S_{\text{sub},*} \times H \times [c_{\text{sub},1.\text{sup},T}, c_{\text{sub},2.\text{sup},T}, c_{\text{sub},3.\text{sup},T}, c_{\text{sub},4.\text{sup},T}].\text{sup},T = 0$ . In particular, helper  $i$ ,  $i \neq *$  transmits

$$(127) S_* \begin{bmatrix} h_{1,i} \\ h_{2,1} \end{bmatrix} c_i,$$

which is 1 symbol in  $\text{custom character}$ , achieving an optimal total repair bandwidth of 3 symbols in IF.

(128) According to embodiments, a flexible MSR code includes every entry in the code array is a vector in  $\text{custom character.sup},2$ . The code array is shown as below, each column being a node

$$(129) \begin{bmatrix} C_{1,1,1} & C_{1,1,2} & C_{1,1,3} & C_{1,1,4} \\ C_{1,2,1} & C_{1,2,2} & C_{1,2,3} & C_{1,2,4} \\ C_{2,1,1} & C_{2,1,2} & C_{2,1,3} & C_{1,1,4} \end{bmatrix}.$$

(130) The code has 2 layers, where  $C_{\text{sub.1},m.\text{sub.1}.sub.i} \in \text{custom character}^{\text{sup.2}}$  are in Layer 1 and  $C_{\text{sub.2},m.\text{sub.2}.sub.i}$  are in Layer 2 with  $m.\text{sub.1}=1,2$ ,  $m.\text{sub.2}=1$ ,  $i \in [4]$ . Each  $C_{\text{sub.j},m.\text{sub.j}.sub.i}$  is the vector  $[c_{\text{sub.j},m.\text{sub.j}.sub.i,1}, c_{\text{sub.j},m.\text{sub.j}.sub.i,2}]^{\text{sup.T}}$  with elements in  $\text{custom character}$ . The code totally contains 48 bits with 24 information bits, and each node contains 12 bits. We define the code with the 3 parity check matrices shown below. Let

$$(131) H_1 = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & h_{1,4} & h_{1,1} \\ h_{2,1} & h_{2,2} & h_{2,3} & h_{2,4} & h_{2,1} \end{bmatrix}, H_2 = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & h_{1,4} & h_{1,2} \\ h_{2,1} & h_{2,2} & h_{2,3} & h_{2,4} & h_{2,2} \end{bmatrix}, H_3 = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & h_{1,4} & h_{1,1} \\ h_{2,1} & h_{2,2} & h_{2,3} & h_{2,4} & h_{2,4} \end{bmatrix}$$

(132) The code is defined by

$$H_{\text{sub.1}} \times [C_{\text{sub.1},1,1,\text{sup.T}}, C_{\text{sub.1},1,2,\text{sup.T}}, C_{\text{sub.1},1,3,\text{sup.T}}, C_{\text{sub.1},1,4,\text{sup.T}}, C_{\text{sub.2},1,1,\text{sup.T}}]^{\text{sup.T}} = 0, \\ H_{\text{sub.2}} \times [C_{\text{sub.1},2,1,\text{sup.T}}, C_{\text{sub.1},2,2,\text{sup.T}}, C_{\text{sub.1},2,3,\text{sup.T}}, C_{\text{sub.1},2,4,\text{sup.T}}, C_{\text{sub.2},1,2,\text{sup.T}}]^{\text{sup.T}} = 0, \\ H_{\text{sub.3}} \times [C_{\text{sub.2},1,1,\text{sup.T}}, C_{\text{sub.2},1,2,\text{sup.T}}, C_{\text{sub.2},1,3,\text{sup.T}}, C_{\text{sub.2},1,4,\text{sup.T}}]^{\text{sup.T}} = 0.$$

(133) According to an exemplary embodiment,  $(n, k, \text{custom character}) = (4; 2; 3)$  flexible MSR code parameterized by  $(k.\text{sub.j}, \text{custom character}.\text{sub.j}) \in \{(3,2), (2,3)\}$ . To check that the code defined by  $H_{\text{sub.1}}$  or  $H_{\text{sub.2}}$  is an  $(5, 2)$  MDS code, and  $H_3$  defines an  $(4; 2)$  MDS code, an index of a failed node may be  $* \in \{1,2,3,4\}$ . For the repair, it is first noted that for  $i=1,2$ .

$$(134) \text{rank}(S_* \begin{bmatrix} h_{1,i} \\ h_{2,i} \end{bmatrix}) = \text{rank}(S_* \begin{bmatrix} h_{1,i} \\ h_{2,i} \end{bmatrix}) = \begin{cases} 2, i = * \\ 1, i \neq * \end{cases}.$$

(135) According to embodiments a repair matrix  $S^*$  as described here can repair a failed node  $*$ :

$$S_{\text{sub.}*} \times H_{\text{sub.1}} \times [C_{\text{sub.1},1,1,\text{sup.T}}, C_{\text{sub.1},1,2,\text{sup.T}}, C_{\text{sub.1},1,3,\text{sup.T}}, C_{\text{sub.1},1,4,\text{sup.T}}, C_{\text{sub.2},1,1,\text{sup.T}}]^{\text{sup.T}} = 0, \\ S_{\text{sub.}*} \times H_{\text{sub.2}} \times [C_{\text{sub.1},2,1,\text{sup.T}}, C_{\text{sub.1},2,2,\text{sup.T}}, C_{\text{sub.1},2,3,\text{sup.T}}, C_{\text{sub.1},2,4,\text{sup.T}}, C_{\text{sub.2},1,2,\text{sup.T}}]^{\text{sup.T}} = 0, \\ S_{\text{sub.}*} \times H_{\text{sub.3}} \times [C_{\text{sub.2},1,1,\text{sup.T}}, C_{\text{sub.2},1,2,\text{sup.T}}, C_{\text{sub.2},1,3,\text{sup.T}}, C_{\text{sub.2},1,4,\text{sup.T}}]^{\text{sup.T}} = 0.$$

(136) For helper  $i \in [4]$ ,  $i \neq *$ , it transmits

$$(137) 0 S_* \begin{bmatrix} h_{1,i} \\ h_{2,i} \end{bmatrix} C_{1,1,i}, S_* \begin{bmatrix} h_{1,i} \\ h_{2,i} \end{bmatrix} C_{1,2,i}, S_* \begin{bmatrix} h_{1,i} \\ -h_{2,i} \end{bmatrix} C_{2,1,i},$$

where  $\beta = \beta$  if  $i=1,2$  and  $\beta=1$  if  $i=3,4$ . Repair of a failed node may also require

$$(138) S_* \begin{bmatrix} h_{1,1} \\ h_{2,1} \end{bmatrix} C_{2,1,1} \text{ and } S_* \begin{bmatrix} h_{1,2} \\ h_{2,2} \end{bmatrix} C_{2,1,2}.$$

For any failed node, one 1 symbol is needed from each of the remaining  $C_{\text{sub.j},m.\text{sub.j}.sub.i}$ , which meets the MSR bound.

(139) In this example, codes in the first layer are not required to be MSR codes, thus resulting in a smaller field. However, the rank condition guarantees the optimal repair bandwidth for the entire code. Also, in our general constructions, we do not require the codes in Layers 1 to  $a-1$  to be MSR codes.

(140) Embodiments are also directed to construction of flexible MSR codes by applying a construction to the vector MSR code and the RS MSR code.

(141) Flexible MSR Codes with Parity Check Matrices

(142) According to embodiments, codes may be defined by parity check matrices and certain choices of the parity check matrices allow for a flexible MSR code. According to embodiments, a second construction is provided. The code, for example, may be defined in some  $\text{custom character}^{\text{sup.L}}$  parameterized by  $(k.\text{sub.j}, \text{custom character}.\text{sub.j})$ ,  $j \in [a]$  such that  $k.\text{sub.j} \cdot \text{custom character}.\text{sub.j} = k.\text{custom character}$ ,  $k.\text{sub.1} > k.\text{sub.2} > \dots > k.\text{sub.a} = k, \text{custom character}.\text{sub.a} = \text{custom character}$ . The parity check matrix for the  $mj$ -th row in Layer  $j \in [a]$  is defined as  $H_{\text{sub.j},m.\text{sub.j}} = [h_{\text{sub.j},m.\text{sub.j}.sub.,1} \dots h_{\text{sub.j},m.\text{sub.j}.sub.,ng.\text{sub.j},m.\text{sub.j}.sub.,1} \dots g_{\text{sub.j},m.\text{sub.j}.sub.,k.\text{sub.j}.sub.-k.\text{sub.a}}]$

where each  $h_{\text{sub.j},m.\text{sub.j}.sub.,n} g_{\text{sub.j},m.\text{sub.j}.sub.,1}$  is an  $rL \times L$  matrix with elements in  $\text{custom character}$ . The  $(n+k.\text{sub.j}-k.\text{sub.a}, k.\text{sub.j})$  MDS code  $m.\text{sub.j}$ -th row in Layer  $j$  is defined by

$$H_{\text{sub.j},m.\text{sub.j}} \times [C_{\text{sub.j},m.\text{sub.j}.sub.,1,\text{sup.T}}, C_{\text{sub.j},m.\text{sub.j}.sub.,1,\text{sup.T}}, \dots, C_{\text{sub.j},m.\text{sub.j}.sub.,1,\text{sup.T}}, C_{\text{sub.j},m.\text{sub.j}.sub.,1,\text{sup.T}}, \dots, C_{\text{sub.j},m.\text{sub.j}.sub.,1,\text{sup.T}}]^{\text{sup.T}} = 0,$$

where  $C_{\text{sub.j},m.\text{sub.j}.sub.,i}$  are the stored code word symbols and  $C'_{\text{sub.j},m.\text{sub.j}.sub.,1}$  are the extra parities. In this construction, when we encode the extra parities into lower layers, we set the code word symbols and the corresponding parity check matrix entries exactly the same. Specifically, for Layers  $j < j' \leq a$  the following may be set as

$$g_{\text{sub.j},x,y} = h_{\text{sub.j}',x',y'}, \\ C'_{\text{sub.j},x,y} = C_{\text{sub.j}',x',y'}$$

(143) Here, for  $x \in [l.\text{sub.j} - l.\text{sub.j} - 1]$ ,  $k.\text{sub.j}' - k.\text{sub.a} + 1 \leq y \leq k.\text{sub.j}' - 1 - k.\text{sub.a}$ , then  $g_{\text{sub.j},x,y}$  corresponds to  $h_{\text{sub.j}',x',y'}$  in Layer  $j'$ , and

$$(144) x' = \text{Math} \cdot \frac{x(k_{j'-1} - k_j) + y}{k_j} \cdot \text{Math} \cdot y' = (x(k_{j'-1} - k_j) + y) \bmod k_j$$

where “mod” denotes the modulo operation.

(145) As a result, the 2 extra parities in Layer 1 are exactly the same as the first 2 symbols in Layer 2 with

$C'_{\text{sub},1,1,1} = C_{\text{sub},2,1,1}$ ,  $g_{\text{sub},1,1,1} = h_{\text{sub},2,1,1} = C_{\text{sub},1,2,1} = C_{\text{sub},2,1,2}$ ,  $g_{\text{sub},2,1,2} = h_{\text{sub},2,1,2}$ .  
(146) The code defined by the second construction is a flexible MSR code when an MDS condition and Rank condition satisfy requirements for MDS codes and the repair matrices can be used for every parity check matrix. When an MDS property is satisfied, the second construction is the same as the first construction described above by defining the MDS codes with parity check matrices. For repair, a repair matrix may be used in each row to repair failed nodes. For example, assuming node  $s_{*} \in [n]$  failed, a repair matrix  $S_{*}$  may be used in each row repair, such that  
 $S_{\text{sub},*} \times H_{\text{sub},j,m,\text{sub},j} \times [C_{\text{sub},j,m,\text{sub},j,\text{sub},1,\text{sup},T}, C_{\text{sub},j,m,\text{sub},j,\text{sub},2,\text{sup},T}, \dots, C_{\text{sub},j,m,\text{sub},j,\text{sub},n,\text{sup},T}, C'_{\text{sub},j,m,\text{sub},j,\text{sub},1,\text{sup},T}, \dots, C'_{\text{sub},j,m,\text{sub},j,\text{sub},k,\text{sub},j,\text{sub},-k,\text{sub},a,\text{sup},T}]_{\text{sup},T} = 0$ .  
(147) The information symbols  $C'_{\text{sub},j,m,\text{sub},j,\text{sub},1} = C'_{\text{sub},j,m,\text{sub},j,\text{sub},k,\text{sub},j,\text{sub},-k,\text{sub},a}$  in the lower layers with the same corresponding parity check sub-matrices can be retrieved from lower levels such that the node can be repaired by  $n-1$  helpers. Only  $L/r$  symbols are needed from each helper in order to achieve the optimal repair bandwidth.  
(148) Assuming the field size,  $|\mathbb{F}| > rn$  and  $\lambda_{\text{sub},i,j} \in \mathbb{F}$ ,  $i \in [n]$ ,  $j = 0, 1, \dots, r-1$  are  $rn$  distinct elements. The parity matrix for the  $(n,k)$  MSR code can be represented as

$$(149) H = \begin{bmatrix} I & I & \dots & I \\ A_1 & A_2 & \dots & A_n \\ \vdots & \vdots & \ddots & \vdots \\ A_1^{r-1} & A_1^{r-1} & \dots & A_n^{r-1} \end{bmatrix},$$

where  $I$  is the  $L \times L$  identity matrix and

$$(150) A_i = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}_{L \times L} e_z e_z^T$$

is a vector of length  $L = r \cdot n$  with all elements 0 except the  $z$ -th element which is equal to 1. The  $r$ -ary expansion of  $z$  may be described as  $z = (z_{\text{sub},n}, z_{\text{sub},n-1}, \dots, z_{\text{sub},1})$ , where  $0 \leq z_{\text{sub},i} \leq r-1$  is the  $i$ -th digit from the right and

$$(151) z = \sum_{i=0}^{r-1} z_i r^i.$$

As such,  $A_{\text{sub},i}$  is an  $L \times L$  diagonal matrix with element  $\lambda_{\text{sub},i,z_{\text{sub},i}}$ . The  $L \times L$  repair matrix  $S_{\text{sub},*}, * \in [n]$  are also defined as:

$$S_{\text{sub},*} = \text{Diag}(D_{\text{sub},*}, D_{\text{sub},*}, \dots, D_{\text{sub},*})$$

with

$$(152) \frac{L}{r} \times L$$

matrix  $D_{\text{sub},*}$  and it is shown that:

$$(153) \text{rank}(S_{*} \begin{bmatrix} I \\ A_i \\ \vdots \\ A_i^{r-1} \end{bmatrix}) = \text{rank} \begin{pmatrix} D_{*} \\ D_{*} A_i \\ \vdots \\ D_{*} A_i^{r-1} \end{pmatrix} = \begin{cases} L, & i = * \\ \frac{L}{r}, & i \neq * \end{cases}.$$

(154) Here, for  $0 \leq x \leq r \cdot n - 1$ ,  $0 \leq y \leq r \cdot n - 1$ , the  $(x, y)$ -th entry of  $D_{\text{sub},*}$  equals 1 if the  $r$ -ary expansion of  $x$  and  $y$  satisfies  $(x_{\text{sub},n-1}, x_{\text{sub},n-1}, \dots, x_{\text{sub},1}) = (y_{\text{sub},n}, y_{\text{sub},n-1}, \dots, y_{\text{sub},i+1}, y_{\text{sub},i-1}, \dots, y_{\text{sub},1})$ , and otherwise it equals 0.

(155) Considering an extended field  $\mathbb{F}^*$  from  $\mathbb{F}$  and denoting  $\mathbb{F}^* \cong \mathbb{F} \setminus \{0\}$ ,  $\mathbb{F}^* \cong \mathbb{F} \setminus \{0\}$ . The  $\mathbb{F}^*$  can be partitioned to

$$(156) t = \frac{|\mathbb{F}^*|}{|\mathbb{F}^*|} = 1$$

cosets  $\{\beta_{\text{sub},1} \mathbb{F}^*, \beta_{\text{sub},2} \mathbb{F}^*, \dots, \beta_{\text{sub},t} \mathbb{F}^*\}$ , for some elements  $\beta_{\text{sub},1}, \beta_{\text{sub},2}, \dots, \beta_{\text{sub},t}$  in  $\mathbb{F}$ . The storage nodes define (the first  $n$  nodes):

$$(157) h_{j,m_j,i} = \begin{bmatrix} I \\ A_i \\ \vdots \\ A_i^{r-1} \end{bmatrix}$$

(158) where  $\beta_{\text{sub},j,m,\text{sub},j}$  is chosen from  $\{\beta_{\text{sub},1}, \beta_{\text{sub},2}, \dots, \beta_{\text{sub},t}\}$ . The additional coefficient may be set as  $\beta_{\text{sub},j,m}$ . Then, the extra parity entries  $g_{\text{sub},j,m,\text{sub},j,\text{sub},i}$  can be obtained and  $A_{\text{sub},i}$  might show in  $H_{\text{sub},j,m,\text{sub},j}$  in several times since the extra parity matrices are the same as the information symbols in lower layers. We choose the additional coefficients as below. Condition 1. In each  $H_{\text{sub},j,m,\text{sub},j}$ , the additional coefficients for the same  $A_{\text{sub},i}$  are distinct. With parity check matrices, Construction 2 is a flexible MSR code.

(159) To calculate the required field size, the disclosure discusses how many additional coefficients are required for our flexible MSR codes satisfying Condition 1. In the following, we propose two possible coefficient assignments. It should be noticed that one might find better assignments with smaller field sizes.

(160) The simplest coefficient assignment assigns different additional coefficients to different rows, i.e.,  $\beta_{\text{sub},j,m,\text{sub},j}$  to Row  $m_{\text{sub},j}$  in Layer  $j$  for the storage nodes (the first  $n$  nodes). By doing so, the parity check matrix  $\beta_{\text{sub},j,m,\text{sub},j} A_{\text{sub},i}$ ,  $j \in [a]$ ,  $m_{\text{sub},j} \in [\mathbb{F}^*]$ ,  $i \in [n]$  will show at most twice in Construction 2, i.e., in Layer  $j$  corresponding to storage Node  $i$ , and in Layer  $j'$  corresponding to an extra parity, for some  $j > j'$ . Hence, the same  $A_{\text{sub},i}$  will correspond to different additional coefficients in the same row and Condition 1 is satisfied. In this case, we need

a field size of  $q$  custom character.

(161) In the second assignment, we assign different additional coefficients in different layers for the storage nodes (the first  $n$  nodes), but for different rows in the same layer, we might use the same additional coefficient. For a given row, the storage nodes will not conflict with the extra parities since the latter correspond to the storage nodes in other layers. Also, the extra parities will not conflict with each other if they correspond to the storage nodes in different layers. Then, we only need to check the extra parities in the same row corresponding to storage nodes in the same layer. For the extra parities/storage nodes  $g_{\text{sub},j,x,y} = h_{\text{sub},j',x',y'}$ , given  $j, x, j', y'$ , the additional coefficients should be different. In this case  $k_{\text{sub},j'} - k_{\text{sub},a+1} \leq y \leq k_{\text{sub},j} - 1 - k_{\text{sub},a}$ , and there will be at most

$$(162) 0 \leq \frac{k_{j-1} - k_j}{k_j} \leq 1$$

that make  $y'$  a constant.

(163) By assigning

$$(164) \beta_{j,x,y} = \frac{k_{j-1} - k_j}{k_j} \beta_{j',x',y'}$$

number of  $\beta$  in Layer  $j'$ ,  $j' \geq 2$  (in Layer 1 we only need one  $\beta$ ), Condition 1 is satisfied.

(165) The total number of required additional coefficients is

$$(166) \sum_{j=2}^{\ell} (k_{j-1} - k_j) \leq t.$$

Since  $(k_{\text{sub},j-1} - k_{\text{sub},j}) \leq k_{\text{sub},j} - k_{\text{sub},j-1}$  then

$$(167) t = 1 + \sum_{j=2}^{\ell} \left[ \frac{k_{j-1} - k_j}{k_j} \right] = 1 + \sum_{j=2}^{\ell} \left[ \frac{\ell_j - \ell_{j-1}}{\ell_{j-1}} \right] \leq 1 + \sum_{j=2}^{\ell} (\ell_j - \ell_{j-1}) \leq \ell.$$

(168) In the best case,  $k_{\text{sub},j-1} - k_{\text{sub},j} \leq k_{\text{sub},j}$  for all  $j$ , the number of coefficients is  $a$ , and  $|C_{\text{sub},j}| \geq a$ .

(169) Compare constructions with another flexible MSR constructions, embodiments provide code such that in each node  $C_{\text{sub},j} \supset C_{\text{sub},j-1}$ , where  $|C_{\text{sub},j}| \geq t(n-k)n$ . Namely, each node requires  $(n-k)n \log_2(t(n-k)n)$  bits. Unlike other codes, configurations here in allow for choosing  $C_{\text{sub},j} = S$ .

(170) Embodiments are also directed to Flexible RS MSR codes including construction of Reed-Solomon (RS) MSR codes.

(171) An  $RS(n, k)$  code over the finite field  $GF(q)$  may be defined as

$$RS(n, k) = \{ (f(\alpha_{\text{sub},1}), f(\alpha_{\text{sub},2}), \dots, f(\alpha_{\text{sub},n})) : f \in GF(q)[x], \deg(f) \leq k-1 \}.$$

where the evaluation points are defined as  $\{\alpha_{\text{sub},1}, \alpha_{\text{sub},2}, \dots, \alpha_{\text{sub},n}\}$ .  $\deg()$  denotes the degree of a polynomial. The encoding polynomial  $f(x) = u_{\text{sub},0} + u_{\text{sub},1}x + \dots + u_{\text{sub},k-1}x^{k-1}$ , where  $u_{\text{sub},i} \in GF(q)$ ,  $i=0,1,\dots,k-1$  are the information symbols. Every evaluation symbol  $f(\alpha_{\text{sub},i})$ ,  $i \in [n]$  is called a code word symbol. RS codes are MDS codes, namely from any  $k$  code word symbols, the information can be recovered.

(172) By letting  $GF(q)$  be the base of field  $GF(q^L)$  such that  $GF(q^L) = GF(q)[x]/(x^L - 1)$ . For repairing RS codes, any linear repair scheme for a given  $RS(n, k)$  over the finite field  $GF(q^L)$  is equivalent to finding a set of repair polynomials  $p_{\text{sub},*}, v(x)$  such that for the failed node  $f(\alpha_{\text{sub},*})$ ,  $* \in [n]$ ,

$$(173) \text{rank}_{GF(q^L)}(\{p_{\text{sub},*}, v(x) : v(x) \in GF(q^L)\}) = L,$$

where the rank is defined as  $\text{rank}_{GF(q^L)}(\{y_{\text{sub},1}, y_{\text{sub},2}, \dots, y_{\text{sub},i}\})$ , as the cardinality of a maximum subset of  $\{y_{\text{sub},1}, y_{\text{sub},2}, \dots, y_{\text{sub},i}\}$  that is linearly independent over  $GF(q^L)$ .

(174) The transmission from helper  $f(\alpha_{\text{sub},i})$  is

$$T_{\text{sub},r} = \frac{f(\alpha_{\text{sub},i})}{\prod_{j \neq i} (x - \alpha_{\text{sub},j})}, x \in GF(q^L),$$

where the trace function  $T_{\text{sub},r} = \frac{f(\alpha_{\text{sub},i})}{\prod_{j \neq i} (x - \alpha_{\text{sub},j})}$  is a linear function such that for all  $x \in GF(q^L)$ ,  $T_{\text{sub},r} \in GF(q^L)$ . The repair bandwidth for the  $i$ -th helper is  $b_{\text{sub},i} = \text{rank}_{GF(q^L)}(\{p_{\text{sub},*}, v(\alpha_{\text{sub},i}) : v \in GF(q^L)\})$ , symbols in  $GF(q^L)$ . The Flexible RS MSR code construction is similar to Construction 2 based on parity check matrices.

(175) According to embodiments, a third construction (construction 3) defines a code in  $GF(q^L)$  with a set of pairs  $(k_{\text{sub},j}, C_{\text{sub},j})$  such that  $k_{\text{sub},j} \leq a$ ,  $C_{\text{sub},j} \in GF(q^L)^{k_{\text{sub},j}}$ ,  $k_{\text{sub},1} > k_{\text{sub},2} > \dots > k_{\text{sub},a} = k$ ,  $C_{\text{sub},a} \in GF(q^L)^k$ ,  $r = n - k$ . In the  $m_j$ -th row in layer  $j \in [a]$  the code word symbols  $C_{\text{sub},j,m_j,i}$ ,  $i \in [n]$  are defined as

$$(176) C_{j,m_j,i} = f_{j,m_j}(\alpha_{\text{sub},j,m_j,i})$$

and the extra parities  $C'_{\text{sub},j,m_j,i}$ ,  $i \in [k_{\text{sub},j} - k_{\text{sub},a}]$  are defined as

$$C'_{\text{sub},j,m_j,i} = f_{\text{sub},j,m_j}(\alpha_{\text{sub},j,m_j,i+n}), \text{ where } \{f_{\text{sub},j,m_j}(\alpha_{\text{sub},j,m_j,i+n})\}, i \in [n+k_{\text{sub},j} - k_{\text{sub},a}], \text{ is a } RS(n+k_{\text{sub},j} - k_{\text{sub},a}, k_{\text{sub},j}) \text{ code.}$$

(177) The encoding polynomial  $f_{\text{sub},j,m_j}(x)$  and the evaluation point  $\alpha_{\text{sub},j,m_j,i}$  may be defined.

(178) In this construction, extra parities and the corresponding evaluation points are set exactly the same as the information symbols in lower layers, and extra parities are arranged in the same way as in Construction 2. Specifically, for  $C'_{\text{sub},j,x,y}$  in Layer  $j$ ,  $x \in [l_{\text{sub},j-1}, l_{\text{sub},j}]$ , when  $k_{\text{sub},j-1} + 1 \leq y \leq k_{\text{sub},j} - k_{\text{sub},j'}$  for  $j+1 \leq j' \leq a$ , it is encoded to Layer  $j'$  with  $\alpha_{\text{sub},j,x,y+n} = \alpha_{\text{sub},j',x',y'}$  and  $C'_{\text{sub},j,x,y} = C_{\text{sub},j',x',y'}$ , with  $x', y'$ . The encoding polynomial  $f_{\text{sub},j',m_j}(x) \in GF(q^L)$  in Layer  $j'$  is defined by the  $k_{\text{sub},j'}$  evaluation points and the code word symbols from the extra parities.

(179) Latency

(180) In this section, we analyze the latency of obtaining the entire information using codes according to embodiments with a flexible number of nodes.

(181) One of the key properties of the flexible storage codes presented in this disclosure is that the decoding rows are the first  $\text{custom character.sub.j}$  rows if we have  $R.\text{sub.j}$  available nodes. As a result, the decoder can simply download symbols one by one from each node, and symbols of Layer  $j$  can be used for Layers  $j, j+1, \dots, a$ . For one pair of  $(R.\text{sub.j}, \text{custom character.sub.j})$ , a random variable  $T.\text{sub.j}$  associated with the time for the first  $R.\text{sub.j}$  nodes transmitting the first  $\text{custom character.sub.j}$  symbols.  $T.\text{sub.j}$  is called the latency for the  $j$ -th layer. Instead of predetermining a fixed pair  $(R, \text{custom character})$  for the system, flexible storage codes allow us to use all possible pairs  $(R.\text{sub.j}, \text{custom character.sub.j}) \in [a]$ . The decoder downloads symbols from all  $n$  nodes and as long as it obtains  $\text{custom character.sub.j}$  symbols from  $R.\text{sub.j}$  nodes, the download is complete. For flexible codes with Layers  $1; 2; \dots; a$ , we use  $T.\text{sub.1,2}, \dots, a = \min(T.\text{sub.j}, j \in [a])$  to represent the latency.

(182) Notice that for the fixed code with the same failure tolerance level, i.e.,  $R = R.\text{sub.a}, \text{custom character} = \text{custom character.sub.a}$ , its latency is  $T.\text{sub.a}$ . Since  $T.\text{sub.1,2}, \dots, a = \min(T.\text{sub.j}, j \in [a]) \leq T.\text{sub.a}$ , and given the storage size per node  $\text{custom character}$ , the number of nodes  $n$ , and recovery threshold  $R = R.\text{sub.a}$ , the flexible storage code can reduce the latency of obtaining the entire information compared to any fixed array code.

(183) Assuming the probability density function (PDF) of  $T.\text{sub.j}$  is  $p.\text{sub.R.sub.j.sub., custom character.sub.j}(t)$ , the expected delay can be calculated as

$$E(T.\text{sub.j}) = \int_0^{\infty} t \cdot p.\text{sub.R.sub.j.sub., custom character.sub.j}(t) dt.$$

(184) If a fixed code is adopted, one can optimize the expected latency and get an optimal pair  $(R^*, \text{custom character}^*)$  for a given distribution. However, a flexible storage code still outperforms such an optimal fixed code in latency due to embodiments described herein and in particular the flexible storage code. Moreover, in practice the choice of  $(n, k, R, \text{custom character})$  depends on the system size and the desired failure tolerance level and is not necessarily optimized for latency.

(185) Regarding Hard Disk Drive (HDD) storage system as an example to calculate the latency of flexible storage codes according to embodiments, additional latency can be saved compared to a fixed MDS code. In this part, we compute the overall latency of a flexible code with  $(R.\text{sub.1}, \text{custom character.sub.1})$ ,  $(R.\text{sub.2}, \text{custom character.sub.2})$ , and length  $n$ . Comparing latency with the latency of fixed codes with  $(n, R.\text{sub.1}, \text{custom character.sub.1})$  and  $(n, R.\text{sub.2}, \text{custom character.sub.2})$ , respectively.

(186) In the HDD latency model, the overall latency consists of the positioning time and the data transfer time. The positioning time measures the latency to move the hard disk arm to the desired cylinder and rotate the desired sector to under the disk head. As the accessed physical address for each node is arbitrary, we assume the positioning time is a random variable uniformly distributed, denoted by  $U(0, t.\text{sub.pos})$ , where  $t.\text{sub.pos}$  is the maximum latency required to move through the entire disk. The data transfer time is simply a linear function of the data size, and we assume the transfer time for a single symbol in our code is  $t.\text{sub.trans}$ . Therefore, the overall latency model is  $X + \text{custom character} \cdot t.\text{sub.trans}$ , where  $X \sim U(0, t.\text{sub.pos})$  and  $\text{custom character}$  is the number of accessed symbols.

(187) Consider an  $(n, R, \text{custom character})$  fixed code. When  $R$  nodes finish the transmission of  $\text{custom character}$  symbols, all the information is obtained. The corresponding latency is called the  $R$ -th order statistics. For  $n$  independent random variables satisfying  $U(0, t.\text{sub.pos})$ , the  $R$ -th order statistics for the positioning time, denoted by  $U_R$ , satisfies a beta distribution:

$$U.\text{sub.R} \sim \text{Beta}(R, n+1-R, -, t.\text{sub.pos}).$$

with expectation

$$(188) E[U_R] = \frac{R}{n+1} t_{\text{pos}}.$$

For a random variable  $Y \sim \text{Beta}(\alpha, \beta, a, c)$ , the probability density function (pdf) is defined as

$$(189) f(Y = y; \alpha, \beta, a, c) = \frac{(y-a)^{\alpha-1} (c-y)^{\beta-1}}{(c-a)^{\alpha+\beta-1} B(\alpha, \beta)}, \text{ where } B(\alpha, \beta) = \int_0^1 t^{\alpha-1} (1-t)^{\beta-1} dt$$

is the beta function.

(190) The expectation of overall latency for an  $(n, R.\text{sub.1}, \text{custom character.sub.1})$  fixed code, denoted by  $T.\text{sub.1}$ , is

$$(191) E[T_1] = \frac{R_1}{n+1} t_{\text{pos}} + \ell_1 t_{\text{trans}}.$$

(192) Similarly, the expected overall latency  $E(T.\text{sub.2})$  for the fixed  $(n, R.\text{sub.2}, \text{custom character.sub.2})$  code is

$$(193) E[T_2] = \frac{R_2}{n+1} t_{\text{pos}} + \ell_2 t_{\text{trans}}.$$

(194) Considering the flexible code with 2 layers, the difference of the positioning times  $U.\text{sub.R.sub.1}$  and  $U.\text{sub.R.sub.2}$  is  $\Delta U = U.\text{sub.R.sub.1} - U.\text{sub.R.sub.2} \sim \text{Beta}(R.\text{sub.1} - R.\text{sub.2}, n+1 - (R.\text{sub.1} - R.\text{sub.2}), 0, t.\text{sub.pos})$ .

(195) The expectation of the overall latency for the flexible code denoted by  $T.\text{sub.1,2}$ , may be denoted as

$$E(T_{1,2}) = E(\min(T_1, T_2))$$

$$= E(T_1 \cdot \text{Math. } T_1 - T_2 \leq 0) P(T_1 - T_2 \leq 0) + E(T_2 \cdot \text{Math. } T_1 - T_2 \leq 0) P(T_1 - T_2 > 0)$$

$$(196) 0 = E(T_1) - E(T_1 - T_2 \cdot \text{Math. } T_1 - T_2 > 0) P(T_1 - T_2 > 0)$$

$$= \frac{R_1}{n+1} t_{\text{pos}} + \ell_1 t_{\text{trans}} - \int_{(\ell_2 - \ell_1) t_{\text{trans}}}^{t_{\text{pos}}} [U - (\ell_2 - \ell_1) t_{\text{trans}}] f(U) dU,$$

where the last term is saved latency compared to an  $(n, R.\text{sub.1}, \text{custom character.sub.1})$  code. The saved latency can be calculated as:

$$(197) \quad E(T_1 - T_{1,2}) = \int_{(\ell_2 - \ell_1)t_{\text{trans}}}^{t_{\text{pos}}} [U - (\ell_2 - \ell_1)t_{\text{trans}}] f(U) dU$$

$$= \frac{at_{\text{pos}}}{a+B} I_{1-x}(b, a+1) - (\ell_2 - \ell_1)t_{\text{trans}} I_{1-x}(b, a), \text{ where } B(\cdot, \cdot) = \int_{t=0}^1 t^{a-1} (1-t)^{b-1} dt$$

is the beta function.

(198) The expectation of overall latency for an (n, R.sub.1, custom character.sub.1) fixed code, denoted by T.sub.1 is

$$(199) \quad E(T_1) = \frac{R_1}{n+1} t_{\text{pos}} + \ell_1 t_{\text{trans}}.$$

(200) Similarly, the expected overall latency E(T.sub.2) for the fixed (n, R.sub.2, custom character.sub.2) code is

$$(201) \quad E(T_2) = \frac{R_2}{n+1} t_{\text{pos}} + \ell_2 t_{\text{trans}}.$$

(202) Considering the flexible code with 2 layers, the difference of the positioning times U.sub.R1 and U.sub.R2 is  $\Delta U = U_{\text{sub.R.sub.1}} - U_{\text{sub.R.sub.2}} = \text{Beta}(R_{\text{sub.1}} - R_{\text{sub.2}}, n+1 - (R_{\text{sub.1}} - R_{\text{sub.2}}), 0, t_{\text{sub.pos}})$ .

(203) As a result, an expectation of the overall latency for flexible code described herein, denoted by T.sub.1,2, may be

$$\begin{aligned} E(T_{1,2}) &= E(\min(T_1, T_2)) \\ &= E(T_1 \cdot \text{Math. } T_1 - T_2 \leq 0) P(T_1 - T_2 \leq 0) + \\ &\quad E(T_2 \cdot \text{Math. } T_1 - T_2 \leq 0) P(T_1 T_2 > 0) \\ (204) \quad &= E(T_1) - E(T_1 - T_2 \cdot \text{Math. } T_1 - T_2 > 0) P(T_1 - T_2 > 0) \\ &= \frac{R_1}{n+1} t_{\text{pos}} + \ell_1 t_{\text{trans}} - \\ &\quad \int_{(\ell_2 - \ell_1)t_{\text{trans}}}^{t_{\text{pos}}} [U - (\ell_2 - \ell_1)t_{\text{trans}}] f(U) dU, \end{aligned}$$

where the last term is saved latency compared to an (n, R.sub.1, custom character.sub.1) code.

(205) The latency of a fixed MDS code is a function of n, R, custom character, t.sub.pos, and t.sub.trans. According to embodiments, one can optimize the code reconstruction threshold R\* based on other parameters. However, the system parameters might change over time and one “optimal” R\* cannot.

(206) FIG. 8 illustrates a graphical representation of latency according to one or more embodiments. Graph 800 includes overall latency of fixed codes as 810 and flexible recoverable codes latency as 805. Parameters may be fixed and the unit data transfer time t.sub.trans may be change. For fixed codes, a smaller R provides a lower latency with a smaller t.sub.trans, and when t.sub.trans grows, a larger R is preferred. However, our flexible code always provides a smaller latency, and can save 2%-5% compared to the better of the two fixed codes

(207) In conclusion, embodiments are directed to flexible storage codes and investigation of the construction of such codes under various settings. Our analysis shows the benefit of our codes in terms of latency.

(208) While this disclosure has been particularly shown and described with references to exemplary embodiments thereof, it will be understood by those skilled in the art that various changes in form and details may be made therein without departing from the scope of the claimed embodiments.

## Claims

1. A method for reconstructing data using flexible storage codes, the method comprising: determining, by a device, a node failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes; determining, by the device, a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures; and reconstructing, by the device, the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible locally recoverable code (LRC), wherein flexible LRC is an array code for finite field of information, the array generated based on a code word length n, dimension k, and code word symbol l, and wherein the code is parametrized by {(R.sub.j, k.sub.j, custom character.sub.j): 1 ≤ j ≤ a} that satisfies k.sub.j, custom character.sub.j = k custom character, 1 ≤ j ≤ a, k.sub.1 > k.sub.2 > ... > k.sub.a = k, custom character.sub.a = custom character and  $R_j = k_j + \frac{k_j}{r} - 1$ , for single node failure recovery from a subset of nodes of the storage code.
2. The method of claim 1, wherein determining a node failure includes identifying at least one node failure from the storage code for the information.
3. The method of claim 1, wherein the plurality storage nodes are in a distributed network and information is received by the device with symbols encoded over a finite field into a number of nodes, the flexible storage code configured for a flexible number of nodes and symbols.
4. The method of claim 1, wherein the storage code is arranged as an array of codes including parameters for code word length, dimension and size of each storage node.
5. The method of claim 1, wherein the storage code is at least one of a maximum distance separable (MDS) storage code and a minimum storage regenerating (MSR) storage code.
6. The method of claim 1, wherein reconstructing the received information with the number of nodes and the number of symbols of a flexible storage code includes using a subarray of nodes and symbols of the storage code for the information.
7. A method for reconstructing data using flexible storage codes, the method comprising: determining, by a device, a node

failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes; determining, by the device, a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures; and reconstructing, by the device, the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible partial maximum distance separable code (PMDSC) wherein, wherein flexible PMDSC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , code word symbol  $l$ , and extra symbol failures  $s$ , and wherein the code is parametrized by a set of  $\{(R_{sub,j}, k_{sub,j}, \text{custom character}_{sub,j}): 1 \leq j \leq a\}$  satisfying  $k_{sub,j}, \text{custom character}_{sub,j} = k_{\text{custom character}}, 1 \leq j \leq a, k_{sub,1} > k_{sub,2} > \dots > k_{sub,a} = k, \text{custom character}_{sub,a} = \text{custom character}$  and  $R_{sub,j} = k_{sub,j}$  such that  $\text{custom character}_{sub,j}$  symbols are accessed in each node.

8. A method for reconstructing data using flexible storage codes, the method comprising: determining, by a device, a node failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes; determining, by the device, a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures; and reconstructing, by the device, the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible minimum storage regenerating code (MSRC), wherein repair bandwidth is a minimum amount of transmission required to repair a single node failure from all remaining nodes normalized by size of the node, wherein the repair bandwidth is bounded by the minimum storage regenerating (MSR) bound as  $b \geq \frac{n-1}{n-k}$  based on a code word length  $n$ , and dimension  $k$ .

9. A device configured to reconstruct data using flexible storage codes, the device comprising: a receiver configured to receive information from a distributed network; and a controller coupled to the receiver, the controller configured to determine a node failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes, determine a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures, and reconstruct the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible locally recoverable code (LRC), wherein flexible LRC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , and code word symbol  $l$ , and wherein the code is parametrized by  $\{(R_{sub,j}, k_{sub,j}, \text{custom character}_{sub,j}): 1 \leq j \leq a\}$  that satisfies  $k_{sub,j}, \text{custom character}_{sub,j} = k_{\text{custom character}}, 1 \leq j \leq a, k_{sub,1} > k_{sub,2} > \dots > k_{sub,a} = k, \text{custom character}_{sub,a} = \text{custom character}$  and  $R_j = k_j + \frac{k_j}{r} - 1$ , for single node failure recovery from a subset of nodes of the storage code.

10. The device of claim 9, wherein determining a node failure includes identifying at least one node failure from the storage code for the information.

11. The device of claim 9, wherein the plurality storage nodes are in a distributed network and information is received by the device with symbols encoded over a finite field into a number of nodes, the flexible storage code configured for a flexible number of nodes and symbols.

12. The device of claim 9, wherein the storage code is arranged as an array of codes including parameters for word length, dimension, and size of each storage node.

13. The device of claim 9, wherein the storage code is at least one of a maximum distance separable (MDS) storage code and a minimum storage regenerating (MSR) storage code.

14. The device of claim 9, wherein reconstructing the received information with the number of nodes and the number of symbols of a flexible storage code includes using a subarray of nodes and symbols of the storage code for the information.

15. A device configured to reconstruct data using flexible storage codes, the device comprising: a receiver configured to receive information from a distributed network; and a controller coupled to the receiver, the controller configured to determine a node failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes, determine a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures, and reconstruct the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible partial maximum distance separable code (PMDSC) wherein, wherein flexible PMDSC is an array code for finite field of information, the array generated based on a code word length  $n$ , dimension  $k$ , code word symbol  $l$ , and extra symbol failures  $s$ , and wherein the code is parametrized by a set of  $\{(R_{sub,j}, k_{sub,j}, \text{custom character}_{sub,j}): 1 \leq j \leq a\}$  satisfying  $k_{sub,j}, \text{custom character}_{sub,j} = k_{\text{custom character}}, 1 \leq j \leq a, k_{sub,1} > k_{sub,2} > \dots > k_{sub,a} = k, \text{custom character}_{sub,a} = \text{custom character}$  and  $R_{sub,j} = k_{sub,j}$  such that  $\text{custom character}_{sub,j}$  symbols are accessed in each node.

16. A device configured to reconstruct data using flexible storage codes, the device comprising: a receiver configured to receive information from a distributed network; and a controller coupled to the receiver, the controller configured to

determine a node failure for received information using a storage code for the information, wherein the storage code is encoded as information symbols into a number of storage nodes, and the received information is received from at least one of a plurality of storage nodes, determine a number of nodes and a number of symbols of a flexible storage code for error correction, wherein the flexible storage code is generated using the storage code for the information and wherein the number of nodes and the number of symbols of the flexible storage code is determined based on a number of node failures, and reconstruct the received information using the determined number of nodes and the number of symbols of the flexible storage code, wherein the flexible storage code is configured as a flexible minimum storage regenerating code (MSRC), wherein repair bandwidth is a minimum amount of transmission required to repair a single node failure from all remaining nodes normalized by size of the node, wherein the repair bandwidth is bounded by the minimum storage regenerating (MSR) bound as  $b \geq \frac{n-1}{n-k}$  based on a code word length  $n$ , and dimension  $k$ .

---